

v2

Threat Hunting Professional

Hunting Malware

Section 03 | Module 03

© Caendra Inc. 2020
All Rights Reserved

Table of Contents

MODULE 03 | HUNTING MALWARE

3.1 Introduction

3.2 Detection Tools

3.3 Detection Techniques

3.4 Memory Analysis

3.5 Malware Analysis



Learning Objectives

By the end of this module, you should have a better understanding of:

- ✓ Malware detection tools
- ✓ Malware detection techniques
- ✓ Memory hunting and analysis
- ✓ The importance of malware analysis



Introduction



3.1 Introduction

Malware is not going anywhere anytime soon. Malware authors use various tools and techniques to remain undetected for as long as possible.

We also need various tools and techniques to hunt for them.



3.1 Introduction

We are hunting for malware in various locations:

- Hiding in plain sight
- Injected into other processes
- In files (macros, for example)
- In email attachments
- In memory (known as Fileless malware)
- Etc.

```
21 # @param: control - the control observation
22 # @param: observations - an array of observations, in memory
23 # @param: experiment - the experiment this result is for
24 # @param: candidates - the candidates to evaluate
25 # @param: control - the control observation
26
27 def initialize(experiment, observations = [], control = null)
28   @experiment = experiment
29   @observations = observations
30   @control = control
31   @candidates = observations - [control]
32   evaluate_candidates
33
34   freeze
35 end
36
37 # @param: the experiment's context
38 def context
39   @context
40 end
41
42 # @param: the name of the experiment
43 def experiment_name
44   @experiment_name
45 end
46
47 # @param: the result a match between all
48 def match
49   @match
50 end
51
52 # @param: result.rb 1.1
```



3.1 Introduction

The tools presented in this module do not represent an exhaustive list by no means, but remember, you're being trained to hunt and to take a proactive approach.

This module will reveal that there are tools available to aid you in your hunts.



Detection Tools



3.2 Detection Tools

In this section, we will look at various tools that will aid us in hunting for malware in our networks.

Whether it's a Meterpreter session or a DLL injection, we should have a plethora of tools at our disposal when we're hunting for specific attack signatures.

```
20 with open('result.txt', 'w') as f:
21     f.write('Result: %s\n' % result)
22
23 # Save the result to a file
24 def save_result(result):
25     with open('result.txt', 'w') as f:
26         f.write('Result: %s\n' % result)
27
28 # Initialize the experiment
29 def initialize(experiment, observations = [], control = None):
30     @experiment = experiment
31     @observations = observations
32     @control = control
33     @candidates = observations - {control}
34     evaluate_candidates()
35
36 # Run the experiment
37 def run_experiment(experiment_name):
38     # Create the experiment
39     experiment = Experiment(experiment_name)
40     # Initialize the experiment
41     initialize(experiment)
42     # Run the experiment
43     run_experiment(experiment_name)
44     # Save the result
45     save_result(result)
46
47 # Run the experiment
48 if __name__ == '__main__':
49     run_experiment('result.txt')
```



3.2.1 Detection Tools – PE Capture

The NoVirusThanks's **PE Capture** tool captures PE files, executables, DLLs, and drivers loaded into the operating system. Any loaded executable (PE, EXE, etc.) is displayed within the GUI, and a copy is saved in the intercepted folder for further analysis.

The copied file is named as the hash value of the file. Additionally, the tool will log execution events to help you easily find a specific PE file that was previously captured. You can download the tool [here](http://www.novirusthanks.org/products/pe-capture/).

3.2.1 Detection Tools – PE Capture

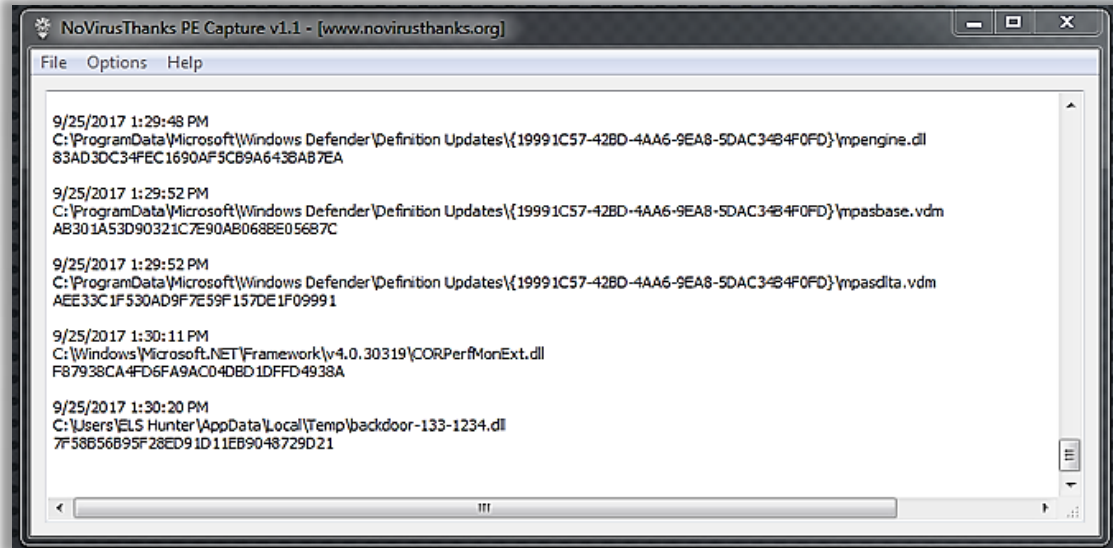
PE Capture is also available in a service-only version.

- This will allow you to install it on multiple PCs.
- It does not have a GUI.
- The program is free for personal use only. You can read more about the tool, and/or download the tool [here](http://www.novirusthanks.org/products/pe-capture-service/).

3.2.1 Detection Tools – PE Capture

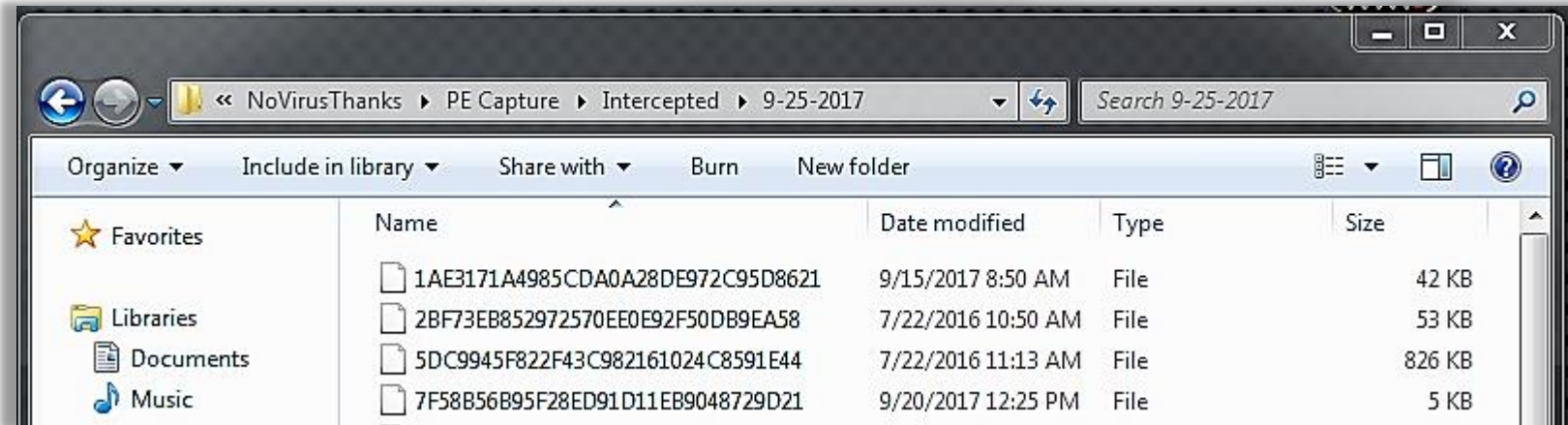
Viewing the screenshot on the right, we can see the suspicious DLL loaded in memory. The GUI shows us the path of the DLL as well as the hash.

We can now look into the Intercepted folder, or the Logs folder, to see what information is saved for us.



3.2.1 Detection Tools – PE Capture

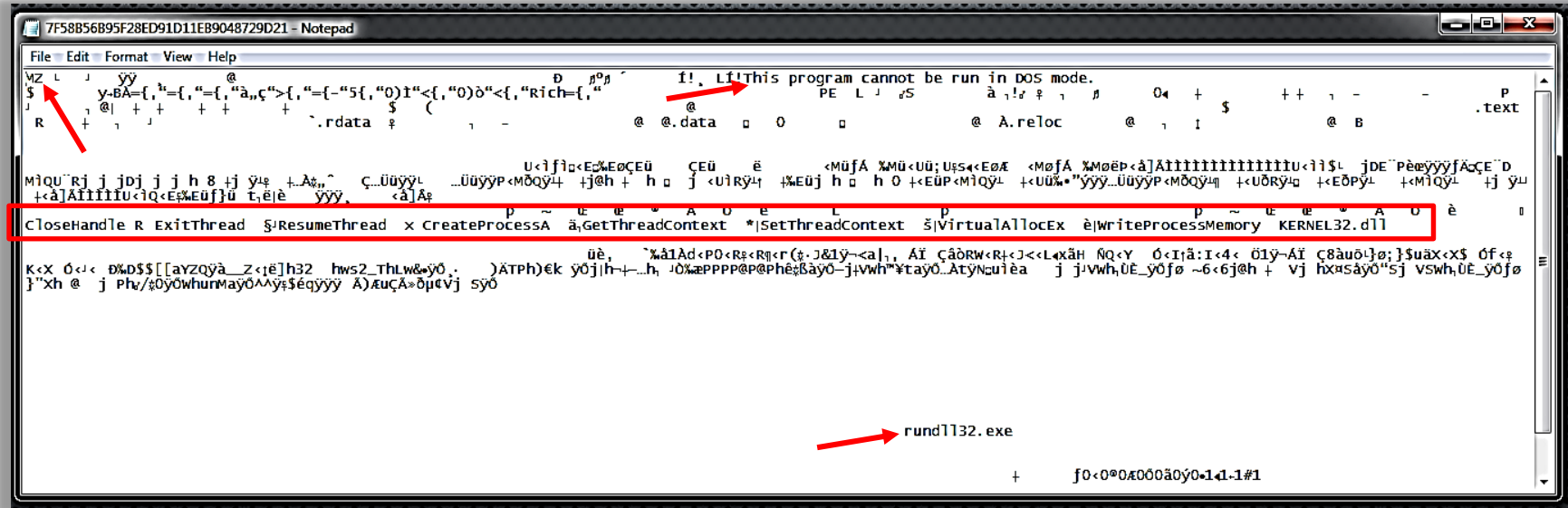
In the File menu, we can either choose **Open “Intercepted” Folder** or **Open “Logs” Folder**.



You can now analyze the exported file to see if it is benign or malicious.

3.2.1 Detection Tools – PE Capture

Based on the indicators shown below, we can already confirm that this is malicious.



7F58B56895F28ED91D11EB9048729D21 - Notepad

File Edit Format View Help

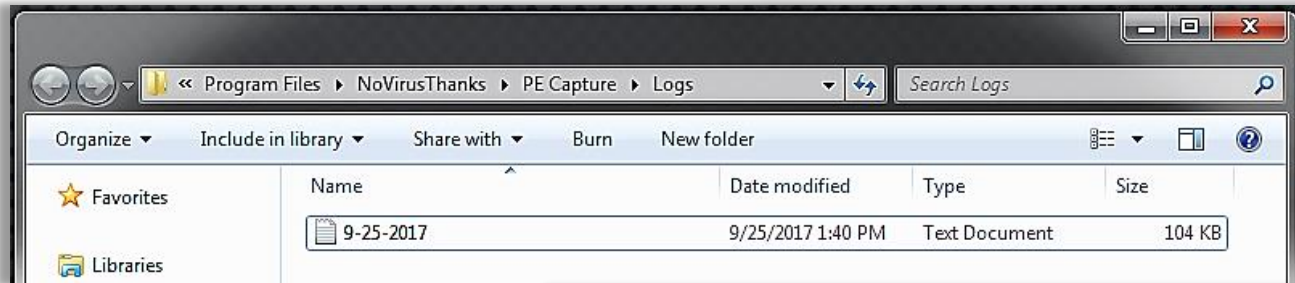
PE L J S This program cannot be run in DOS mode.

CloseHandle R ExitThread \$ResumeThread x CreateProcessA \$GetThreadContext *|SetThreadContext \$|VirtualAllocEx \$WriteProcessMemory KERNEL32.dll

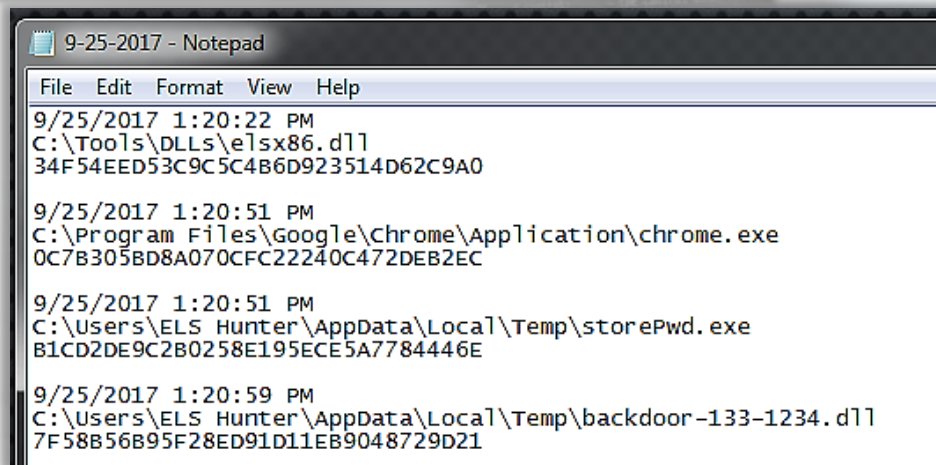
rundll32.exe

f0<0*0A000ä0y0<1d<1#1

3.2.1 Detection Tools – PE Capture



Reviewing the logs is useful to determine what was loaded onto the system earlier that day. You might catch something that you didn't know was loaded since the GUI is displaying information in real time.



3.2.2 Detection Tools – ProcScan.rb

ProcScan, which is written in Ruby, can be used to scan process memory looking for code injection. Unfortunately, it only works for 32-bit systems? applications? and does not support 64-bit systems/applications. You can download the tool [here](#).

To run the tool, type the following command: **ruby ProcScan.rb**

```
C:\Users\elslabs\Desktop\ProcScan>ruby ProcScan.rb
** WARN: Scan all processes? This might take some time. Proceed (yes/no)? : yes
```

3.2.2 Detection Tools – ProcScan.rb

Here is the output of the command if it finds code injection:

```
[+] Scanning: C:\Windows\system32\rundll32.exe  
[+] Scanning Thread Id: 2516 EIP: 0x7fe64f4  
[+] Scanning Thread Stack Frame (Tid: 2516)  
[+] Possibly Injected Code Found at StackFrame[2] ThreadId: 2516 Addr: 0x014b2f2b  
[+] Possibly Injected Code Found at StackFrame[3] ThreadId: 2516 Addr: 0x014b3622  
[+] Possibly Injected Code Found at StackFrame[4] ThreadId: 2516 Addr: 0x014b229c  
[+] Possibly Injected Code Found at StackFrame[5] ThreadId: 2516 Addr: 0x014b1af0  
[+] Possibly Injected Code Found at StackFrame[6] ThreadId: 2516 Addr: 0x014b278f  
[+] Possibly Injected Code Found at StackFrame[7] ThreadId: 2516 Addr: 0x014b280a  
[+] Possibly Injected Code Found at StackFrame[8] ThreadId: 2516 Addr: 0x0153865f  
[+] Possibly Injected Code Found at StackFrame[9] ThreadId: 2516 Addr: 0x015385e6  
[+] Possibly Injected Code Found at StackFrame[10] ThreadId: 2516 Addr: 0x003b0023  
[+] Possibly Injected Code Found at StackFrame[11] ThreadId: 2516 Addr: 0x000e9c2f  
[+] Possibly Injected Code Found at StackFrame[12] ThreadId: 2516 Addr: 0xffffffff  
ff859710ac  
[+] Scanning Thread Id: 1048 EIP: 0x7fe64f4  
[+] Scanning Thread Stack Frame (Tid: 1048)  
[+] Scanning Thread Id: 3592 EIP: 0x7fe64f4  
[+] Scanning Thread Stack Frame (Tid: 3592)
```

3.2.2 Detection Tools – ProcScan.rb

The tool is alerting us that there is possible code injection within thread id 2516 of the rundll32 process.

Unfortunately, the tool doesn't give us the PID within the same output, but this can easily be obtained using PowerShell.

3.2.2 Detection Tools – ProcScan.rb

Simply type **get-process** or **ps** (alias) to retrieve a list of the processes running on the system.

```
PS C:\Users\els labs\Desktop> ps
```

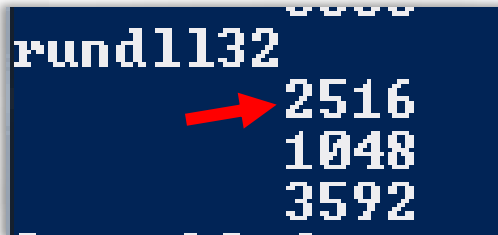
Handles	NPM(K)	PM(K)	WS(K)	UM(M)	CPU(s)	Id	ProcessName
22	2	1800	2268	31	0.00	3156	cmd
50	3	1640	7372	50	0.05	1836	conhost
78	4	1280	8324	71	0.69	2324	conhost
49	3	880	3824	47	0.00	3992	conhost
51	3	1680	6616	50	1.70	4080	conhost
536	5	1292	3988	35	0.48	328	csrss
308	8	10020	5092	53	4.29	380	csrss
189	8	2920	6668	35	1.15	1336	dllhost
144	8	87452	109160	168	27.86	2732	dwm
862	27	39368	77004	253	18.06	2756	explorer
0	0	0	24	0		0	Idle
565	10	2708	6084	32	4.24	496	lsass
139	4	1152	3004	14	0.05	504	lsm
143	9	2396	5192	40	0.22	2092	msdtc
77	4	1536	4544	58	0.03	3368	notepad
360	11	39660	42288	173	1.06	2788	powershell
313	10	22736	33540	160	1.17	4072	powershell
338	16	85400	94864	292	5.19	2384	powershell_ise
142	6	12960	8964	116	0.05	3844	PresentationFontCache
240	7	4864	7292	57	0.04.24	2980	rundll32

3.2.2 Detection Tools – ProcScan.rb

You can also confirm the thread id of the process using PowerShell.

You can use the following command:

```
ps | % {$_.Name ; $_.Threads} | % {"`t{0}" -f $_.ID}}
```



```
rundll32
2516
1048
3592
```

```
21 def initialize
22   @context = context
23   @experiment = experiment
24   @observations = observations
25   @control = control
26   @candidates = candidates
27   evaluate_candidates
28 end
29
30 def initialize(experiment, observations = [], control = nil)
31   @experiment = experiment
32   @observations = observations
33   @control = control
34   @candidates = observations - [control]
35   evaluate_candidates
36 end
37
38 def context
39   experiment.context
40 end
41
42 def initialize_the_experiment
43   # Initialize the name of the experiment
44   def experiment_name
45     experiment.name
46   end
47 end
48
49 def initialize_the_results_a_match_between_obs_and_exp
50   def match
51     ProcScan.result.rb 11
52   end
53 end
```

3.2.3 Detection Tools – Meterpreter Payload Detection

The next tool is called **Meterpreter Payload Detection**. As the name of the tool implies, it will scan all the running processes on the system to detect Meterpreter.

You can download the tool [here](https://github.com/DamonMohammadbagher/Meterpreter_Payload_Detection).

3.2.3 Detection Tools – Meterpreter Payload Detection

You run the tool by simply executing the binary from an elevated command prompt.

```
c:\Tools\Meterpreter_Payload_Detection_v1.0.0.5>Meterpreter_Payload_Detection.exe

[##] Meterpreter Payload Detection
[##] IDS-IPS Version: 1.0.0.5
[##] Console version Published by Damon Mohammadbagher
[##] API code and Meterpreter Signature by Rohan Vazarkar, David Bitner
[##] 9/25/2017 2:21:40 PM Started time
[##] IDS Mode only

Scanning 44 process
14:21:40.4548902 : 1600 vntoolsd is OK
14:21:40.6108904 : 3260 PECapture is OK
14:21:40.9072910 : 3732 PresentationFontCache is OK
14:21:41.1724914 : 1328 suchost is OK
14:21:41.2816916 : 704 vmacthlp is OK
14:21:41.5000920 : 524 lsass is OK
14:21:42.1084931 : 2748 SearchIndexer is OK
14:21:42.2176933 : 2656 notepad is OK
14:21:42.3112934 : 2208 cmd is OK
14:21:42.4516937 : 516 services is OK
```

3.2.3 Detection Tools – Meterpreter Payload Detection

Similar to **PE Capture**, it's a live capture, so the tool will continually run and alert you of a Meterpreter session in memory, as long as that Meterpreter session is active.



3.2.3 Detection Tools – Meterpreter Payload Detection

Here is the output from the tool if it detects a running Meterpreter session in memory.

```
Process BaseAddress : 131072
14:32:15.7192060
Warning : Meterpreter Process Found in Memory !!!
Infected Process: rundll32 : 3808
Process EntryPointAddress : 137112

Infected Memory bytes :
8C-8B-9B-9E-8F-96-A0-8C-86-8C-A0-8F-8D-90-9C-9A-8C-8C-A0-98-
9A-8B-8F-96-9B-FF-FF-FF-8C-8B-9B-9E-8F-96-A0-8C-86-8C-A0-8F-
8D-90-9C-9A-8C-8C-A0-98-9A-8B-A0-96-91-99-90-FF-8C-8B-9B-9E-
8F-96-A0-8C-86-8C-A0-8F-8D-90-9C-9A-8C-8C-A0-88-9E-96-8B-FF-
8C-8B-9B-9E-8F-96-A0-8C-86-8C-A0-8F-8D-90-9C-9A-8C-8C-A0-96-

Process Arguments :
rundll32.exe
Process Thread ID: 3812
Tid StartAddress: 0
Process Thread ID: 3620
Tid StartAddress: 2007723096
Process Thread ID: 3604
Tid StartAddress: 2007723096
Infected Process should be killed : rundll32
Infected Process path : C:\Windows\system32\rundll32.exe
```

3.2.3 Detection Tools – Meterpreter Payload Detection

NOTE: Don't be confused by the Thread ID displayed in the output. This process is not the same as the one shown in the **PE Capture** snapshots.

3.2.4 Detection Tools – Reflective Injection Detection

This tool was created to detect reflective DLL injections running in memory by looking for a PE header. The program also dumps what it finds concerning the injected process, as well as other unlinked executable pages to the root folder.

You can download the tool [here](https://github.com/papadp/reflective-injection-detection).

3.2.4 Detection Tools – Reflective Injection Detection

You run the tool by simply running the binary from an elevated command prompt.

```
c:\Tools\Reflective-Injection-Detection>reflective-injection-detection.exe
no access to memory at 0x00301000 - 0x00340000 in 3336
no access to memory at 0x014d7000 - 0x014f0000 in 3336
no access to memory at 0x07f64000 - 0x07ffa0000 in 3336
no access to memory at 0x7ff41000 - 0x7ff50000 in 3336
no access to memory at 0x7ff51000 - 0x7ffa0000 in 3336
PE header found at 0x00870000 in 3808.
PE header found at 0x008a0000 in 3808.
PE header found at 0x01510000 in 3808.
PE header found at 0x01740000 in 3808.
no access to memory at 0x00731000 - 0x00770000 in 3544
16 pages found.
```

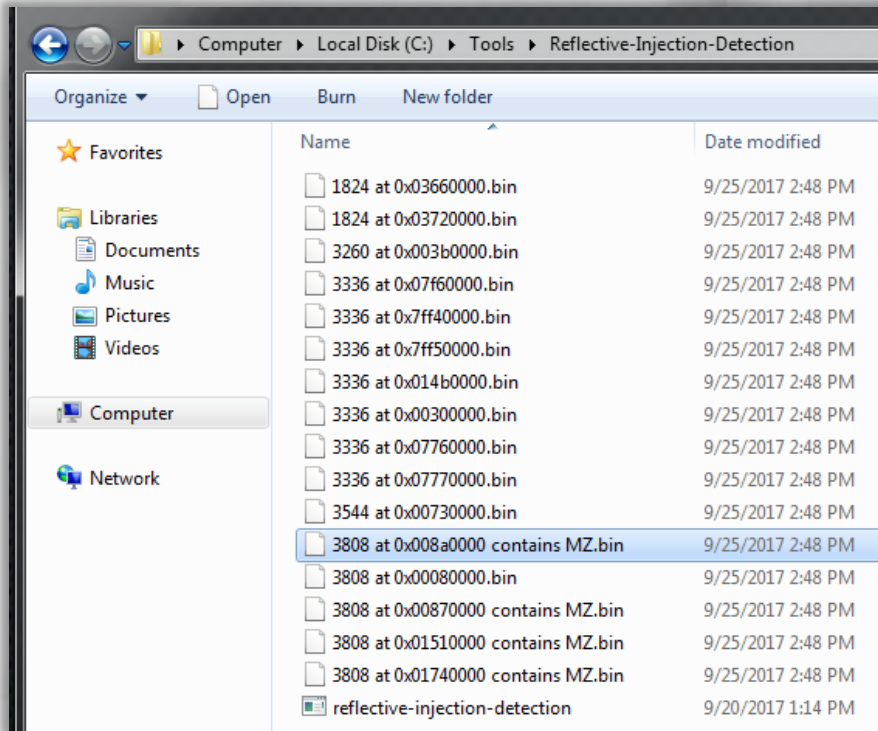


3.2.4 Detection Tools – Reflective Injection Detection

If we navigate to the root folder of the Tool, we will find everything the tool dumped for us, so we can further analyze the artifacts.

Each of the files are named as the PID. This will allow us to easily correlate the file with its process.

You may also notice how the files dumped from process 3808 indicate that 'MZ' was found by listing that information within the name of the dump.



3.2.5 Detection Tools – PowerShell Arsenal

*“**PowerShellArsenal** is a PowerShell module used to aid a reverse engineer. The module can be used to disassemble managed and unmanaged code, perform .NET malware analysis, analyze/scrape memory, parse file formats and memory structures, obtain internal system information, etc.”*



3.2.5 Detection Tools – PowerShell Arsenal

Remember that with the previous tool, **Reflective Injection Detection**, the output gives us the base address and the PID.

To link the output from that tool and the output from PowerShell Arsenal, we will run Reflective Injection Detection again and capture the output. Afterward, we will run the cmdlet **Find-ProcessPEs** from PowerShell Arsenal.

```
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```



3.2.5 Detection Tools – PowerShell Arsenal

Output from Reflective Injection Detection

```
c:\Tools\Reflective-Injection-Detection>reflective-injection-detection.exe
no access to memory at 0x003a1000 - 0x003e0000 in 2472
no access to memory at 0x005f1000 - 0x00630000 in 2472
no access to memory at 0x04195000 - 0x041b0000 in 2472
no access to memory at 0x7ff41000 - 0x7ff50000 in 2472
no access to memory at 0x7ff51000 - 0x7ff58000 in 2472
no access to memory at 0x7ff59000 - 0x7ffa0000 in 2472
PE header found at 0x004f0000 in 3624.
PE header found at 0x00520000 in 3624.
PE header found at 0x00660000 in 3624.
PE header found at 0x007d0000 in 3624.
15 pages found.
```

The suspicious process is ID? PID? 3624 and we see 4 base addresses displayed in the output. Now, let's run Find-ProcessPEs and compare the output.

3.2.5 Detection Tools – PowerShell Arsenal

In this case, the syntax is: **Find-ProcessPEs -ProcessID 3624**

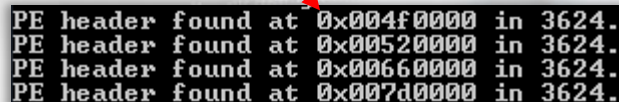
```
PS C:\Tools> Find-ProcessPEs -ProcessID 3624
```

```
ProcessId      : 3624
BaseAddress    : 0x000000000020000
ModuleName     :
Bits          : 32
DOSHeader     : PE_IMAGE_DOS_HEADER
NTHeader      : PE_IMAGE_NT_HEADERS
SectionHeaders : [.rsrc]
ImportDirectory :
Imports       :
ExportDirectory :
Exports       :

ProcessId      : 3624
BaseAddress    : 0x0000000000150000
ModuleName     : C:\Windows\system32\rundll32.exe
Bits          : 32
DOSHeader     : PE_IMAGE_DOS_HEADER
NTHeader      : PE_IMAGE_NT_HEADERS
SectionHeaders : [.text, .data, .rsrc, .reloc]
ImportDirectory : [{ForwarderChain=0; OriginalFirstThunk=16840; TimeDateStamp=0; Name=KERNEL32.dll; FirstThunk=4096}, {ForwarderChain=0; OriginalFirstThunk=17048; TimeDateStamp=0; Name=USER32.dll; FirstThunk=4304}, {ForwarderChain=0; OriginalFirstThunk=17112; TimeDateStamp=0; Name=advapi32.dll; FirstThunk=368}, {ForwarderChain=0; OriginalFirstThunk=17196; TimeDateStamp=0; Name=imaghlp.dll; FirstThunk=4452}, ...]
Imports       : [{FT=198606644; OFT=17240; FunctionName=HeapSetInformation; RVA=198606644; Ordinal=; ModuleName=KERNEL32.dll}, {FT=1986014934; OFT=17262; FunctionName=QueryActiveX; RVA=1986014934; Ordinal=; ModuleName=KERNEL32.dll}, {FT=1986062984; OFT=17278; FunctionName=CloseHandle; RVA=1986062984; Ordinal=; ModuleName=KERNEL32.dll}, {FT=1986070573; OFT=17292; FunctionName=SetFilePointer; RVA=1986070573; Ordinal=; ModuleName=KERNEL32.dll}, ...]
ExportDirectory :
Exports       :
```

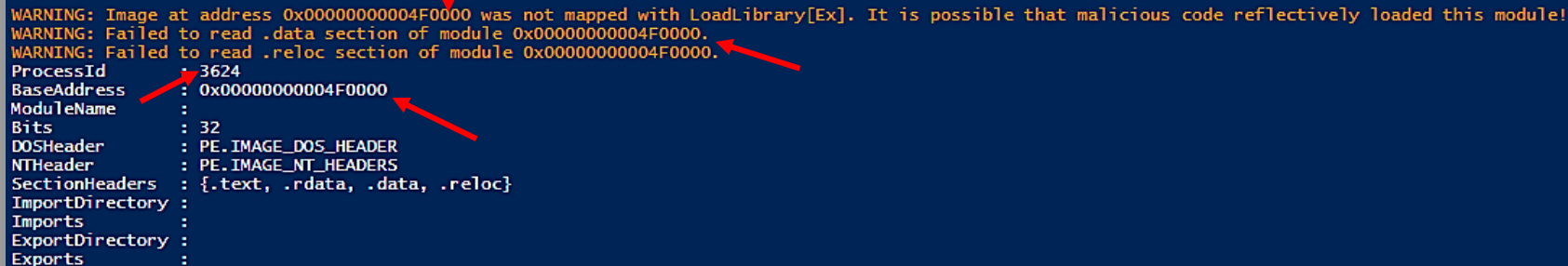
3.2.5 Detection Tools – PowerShell Arsenal

We see that **Find-ProcessPEs** also gives us the same information as far as the base address, but this cmdlet also gives us a bit more.



```
PE header found at 0x004f0000 in 3624.  
PE header found at 0x00520000 in 3624.  
PE header found at 0x00660000 in 3624.  
PE header found at 0x007d0000 in 3624.
```

A red arrow points to the first line of the output.



```
WARNING: Image at address 0x00000000004F0000 was not mapped with LoadLibrary[Ex]. It is possible that malicious code reflectively loaded this module!  
WARNING: Failed to read .data section of module 0x00000000004F0000.  
WARNING: Failed to read .reloc section of module 0x00000000004F0000.  
ProcessId      : 3624  
BaseAddress    : 0x00000000004F0000  
ModuleName     :  
Bits           : 32  
DOSHeader      : PE.IMAGE_DOS_HEADER  
NTHeader       : PE.IMAGE_NT_HEADERS  
SectionHeaders : { .text, .rdata, .data, .reloc }  
ImportDirectory :  
Imports        :  
ExportDirectory :  
Exports        :
```

Red arrows point to the BaseAddress, ModuleName, and SectionHeaders fields.

3.2.5 Detection Tools – PowerShell Arsenal

Here we can see detailed output for the next detected PE.

```
PE header found at 0x004f0000 in 3624.  
PE header found at 0x00520000 in 3624.  
PE header found at 0x00660000 in 3624.  
PE header found at 0x007d0000 in 3624.
```

```
WARNING: Image at address 0x0000000000520000 was not mapped with LoadLibrary[Ex]. It is possible that malicious code reflectively loaded this module!
ProcessId      : 3624
BaseAddress    : 0x0000000000520000
ModuleName     : metsrv.dll
Bits          : 32
DOSHeader     : PE.IMAGE_DOS_HEADER
NTHeader      : PE.IMAGE_NT_HEADERS
SectionHeaders : { .text, .rdata, .data, .reloc }
ImportDirectory : {@{ForwarderChain=0; OriginalFirstThunk=158588; TimeDateStamp=0; Name=WS2_32.dll; FirstThunk=135832}, {@{ForwarderChain=0; OriginalFirstThun
OriginalFirstThunk=158548; TimeDateStamp=0; Name=WININET.dll; FirstThunk=135792}, {@{ForwarderChain=0; OriginalFirstThunk=158492; TimeDa
Imports        : {@{FT=2009819174; OFT=-2147483632; FunctionName=RVA=2009819174; Ordinal=16; ModuleName=WS2_32.dll}, {@{FT=2009818783; OFT=-2147483630; Ord
FunctionName=RVA=2009804119; Ordinal=14; ModuleName=WS2_32.dll}, {@{FT=2009806809; OFT=-2147483536; FunctionName=RVA=2009806809; Ord
ExportDirectory : {@{AddressOfNames=155516; NumberOfNames=81; AddressOfFunctions=155192; Base=1; MinorVersion=0; Name=metsrv.dll; Characteristics=0; Addre
Exports        : {@{ForwardedName=RVA=31149; Ordinal=81; FunctionName=_scheduler_waitable_thread@4}, {@{ForwardedName=RVA=30960; Ordinal=80; Function
FunctionName=scheduler_insert_waitable}, {@{ForwardedName=RVA=30597; Ordinal=78; FunctionName=scheduler_initialize}...
```


3.2.5 Detection Tools – PowerShell Arsenal

Here we can see detailed output for the next detected PE.

```
PE header found at 0x004f0000 in 3624.  
PE header found at 0x00520000 in 3624.  
PE header found at 0x00660000 in 3624.  
PE header found at 0x007d0000 in 3624.
```

```
WARNING: Image at address 0x0000000007D0000 was not mapped with LoadLibrary[Ex]. It is possible that malicious code reflectively loaded this module!
ProcessId      : 3624
BaseAddress    : 0x0000000007D0000
Module Name    : ext_server_stdapi.x86.dll
Bits          : 32
DOSHeader     : PE.IMAGE_DOS_HEADER
NTHeader      : PE.IMAGE_NT_HEADERS
SectionHeaders : { .text, .rdata, .data, .rsrc... }
ImportDirectory : @{{ForwarderChain=0; OriginalFirstThunk=297496; TimeDateStamp=0; Name=PSAPI.DLL; FirstThunk=258872}, @{{ForwarderChain=0; OriginalFirstThun
OriginalFirstThunk=296828; TimeDateStamp=0; Name=IPHLPAPI.DLL; FirstThunk=258204}, @{{ForwarderChain=0; OriginalFirstThunk=297512; Time
Imports        : @{{FT=1977750732; OFT=297804; FunctionName=EnumDeviceDrivers; RVA=1977750732; Ordinal=; ModuleName=PSAPI.DLL}, @{{FT=1977750804; OFT=29
@{{FT=1977751134; OFT=297852; FunctionName=GetDeviceDriverFileNameW; RVA=1977751134; Ordinal=; ModuleName=PSAPI.DLL}, @{{FT=1930714094; OFT=29
ExportDirectory : @{{AddressOfNames=296280; NumberOfNames=4; AddressOfFunctions=296264; Base=1; MinorVersion=0; Name=ext_server_stdapi.x86.dll; Character
Exports         : @{{ForwardedName=; RVA=107963; Ordinal=4; FunctionName=_ReflectiveLoader@0}, @{{ForwardedName=; RVA=109149; Ordinal=3; FunctionName=Ini
@{{ForwardedName=; RVA=109177; Ordinal=1; FunctionName=DeinitServerExtension}}
```

3.2.6 Detection Tools – Get-InjectedThread.ps1

This PowerShell tool can aid you on the hunt to detect code injection. This tool will scan active threads on the system. It will retrieve the starting address of certain functions, such as [NTQueryInformationThread](#), and if executable code is found, it will flag it as injected.

You can download the script [here](#).

3.2.6 Detection Tools – Get-InjectedThread.ps1

We will run this tool against the same suspicious process, rundll32 (PID 3624).

We recommend you conduct independent research to fully understand the output from this tool.

```
PS C:\Tools> Get-InjectedThread

ProcessName      : rundll32.exe
ProcessId        : 3624
Path             : C:\Windows\system32\rundll32.exe
KernelPath       : C:\Windows\System32\rundll32.exe
CommandLine     : rundll32.exe
PathMismatch     : False
ThreadId        : 1288
AllocatedMemoryProtection : PAGE_EXECUTE_READWRITE
MemoryProtection : PAGE_EXECUTE_READWRITE
MemoryState      : MEM_COMMIT
MemoryType       : MEM_PRIVATE
BasePriority      : 4
IsUniqueThreadToken : False
Integrity        : MEDIUM_MANDATORY_LEVEL
Privilege        : SeChangeNotifyPrivilege
LogonId          : 999
SecurityIdentifier : S-1-5-21-1196709796-4176271787-2474106595-1000
Username         : WIN-HIHUBJ9997R\SYSTEM
LogonSessionStartTime : 9/25/2017 6:32:47 PM
LogonType        : System
AuthenticationPackage : NTLM
BaseAddress      : 8656005
Size             : 143360
Bytes            : {85, 139, 236, 131...}
```

Detection Techniques



3.3 Detection Techniques

In this section, we will discuss various techniques to hunt for malware within your network.

Malware authors will try various techniques to ensure that their malware remains undetected. Most of the time, however, the malware in the wild is reused from other malware. This reused malware might be recompiled using a different compiler or modified to remove/add different functionality. In either case, there are techniques to aid us in this hunt.



3.3 Detection Techniques

We will also look at:

- **Fuzzy hashing** and **import hashing** detection techniques to hunt for malware that is reused and is part of an already defined malware family.
- How to detect malware that was already executed on a machine and to correlate various actions that took place on the machine near the time of execution.

3.3.1 Detection Techniques – Fuzzy Hashing

Fuzzy Hashing is a technique where a program, such as [SSDeep](https://ssdeep-project.github.io/ssdeep/index.html), computes context triggered piecewise hashes (CTPH). This technique:

- Can match inputs that have sequences of identical bytes in the same order, although bytes in between the sequences may be different in both content and length.
- Will divide the file into smaller pieces and examine those smaller pieces rather than the file as a whole.

3.3.1 Detection Techniques – Fuzzy Hashing

Virus Total uses **SSDeep**, which performs fuzzy hashing, against files that are uploaded to the platform. The output from **SSDeep** is displayed when the analysis of the uploaded file has completed.

Basic Properties ⓘ

MD5	6c50302984429fd56779bdb38c932ff9
SHA-1	5e66a9df5b7b3271929d977983c0f1486758869e
Authentihash	39ab5f6aeb8aefb2e3bb7672122cdf4e40a8beaff7a260e48b3183ceac098a39
Imphash	f34d5f2d4577ed6d9ceec516c1f5a744
File Type	Win32 EXE
Magic	PE32 executable for MS Windows (GUI) Intel 80386 Mono/.Net assembly
SSDeep	3072:mxM+ImsolAlrRuw+mqv9j1MWLQNMtmmsolNlrRuw+mqv9j1MWLQA:D+IDAA/TmDAN
TrID	Generic CIL Executable (.NET, Mono, etc.) (63.1%) Win64 Executable (generic) (23.8%) Win32 Dynamic Link Library (generic) (5.6%) Win32 Executable (generic) (3.8%) Generic Win/DOS Executable (1.7%)
File Size	207.5 KB

3.3.1 Detection Techniques – Fuzzy Hashing

You can read more about this technique in a paper released by the Digital Forensic Research Workshop [here](#). You can also read about an example usage of SSDeep [here](#).

SSDeep is available on GitHub, [here](#).

http://dfrws.org/sites/default/files/session-files/paper-identifying_almost_identical_files_using_context_triggered_pieewise_hashing.pdf

[https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-\(similarity-matching\).html](https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-(similarity-matching).html)

<https://github.com/ssdeep-project/ssdeep>

```
18 def initialize(experiment, observations = [], control = null)
19   @experiment = experiment
20   @observations = observations
21   @control = control
22   @candidates = observations + [control]
23   evaluate_candidates
24
25   freeze
26 end
27
28 # PUBLIC: the experiment's context
29 def context
30   @experiment.context
31 end
32
33 # PUBLIC: the name of the experiment
34 def experiment_name
35   @experiment.name
36 end
37
38 # PUBLIC: get the results of a search session
39 def search(query)
```

3.3.2 Detection Techniques – Import Hashing

The “**imphash**” technique has been coined by Mandiant, and it is yet another technique implemented by Virus Total. It’s part of the output report displayed when a sample has been analyzed, similar to SSDeep.

Basic Properties ⓘ	
MD5	6c50302984429fd56779bdb38c932ff9
SHA-1	5e66a9df5b7b3271929d977983c0f1486758869e
Authentihash	39ab5f6aeb8aefb2e3bb7672122cfd4e40a8beaff7a260e48b3183ceac098a39
Imphash	f34d5f2d4577ed6d9ceec516c1f5a744
File Type	Win32 EXE
Magic	PE32 executable for MS Windows (GUI) Intel 80386 Mono/.Net assembly
SSDeep	3072:mxM+lmsolAlrRuw+mqv9j1MWLQNMtmmsolNlrRuw+mqv9j1MWLQA:D+IDAA/TmDAN
TRiD	Generic CIL Executable (.NET, Mono, etc.) (63.1%) Win64 Executable (generic) (23.8%) Win32 Dynamic Link Library (generic) (5.6%) Win32 Executable (generic) (3.8%) Generic Win/DOS Executable (1.7%)
File Size	207.5 KB

3.3.2 Detection Techniques – Import Hashing

*“One unique way that Mandiant tracks specific threat groups' backdoors is to track portable executable (PE) imports. Imports are the functions that a piece of software (in this case, the backdoor) calls from other files (typically various DLLs that provide functionality to the Windows operating system). To track these imports, Mandiant creates a hash based on library/API names and their specific order within the executable. We refer to this convention as an **"imphash"** (for **"import hash"**).”*

3.3.2 Detection Techniques – Import Hashing

*“Because of the way a PE's import table is generated (and therefore how its **imphash** is calculated), we can use the **imphash** value to identify related malware samples. We can also use it to search for new, similar samples that the same threat group may have created and used.”*

You can read more about this technique [here](#) and an open source tool to generate PE Import Hashes [here](#).

3.3.3 Detection Techniques – Execution Tracing

If you're familiar with forensics, then you know about the ShimCache. The Windows ShimCache was created to track compatibility issues with executed programs and stores various file metadata. You can read more about the ShimCache [here](#).

Five years ago, Mandiant released a tool called ShimCacheParser to gather this metadata within Windows machines to aid them in their investigations. The tool can be downloaded [here](#).

3.3.3 Detection Techniques – Execution Tracing

This year, they released an updated tool called AppCompatProcessor, and it contains some analytics to look at the execution trace artifacts obtained from AppCompat / AmCache metadata.

```
AppCompatProcessor (Beta 0.8.0 [2017-03-18T13:25:47])
Feedback, bugs, complaints: matias.bevilacqua@mandiant.com

positional arguments:
  database_file          The database to create or work with
  {load,status,list,dump,search,fsearch,filehitcount,tcrr,ptcorr,tstomp,tstack,stack,leven,reconscan,precon,fevil,hashsearch,
h,testset}
  load                  Load (or add new) AppCompat / AmCache data
  status                Print status of database
  list                  List hosts in database
  dump                  Recreate AppCompat/AmCache dump for a given host
  search                Search module
  fsearch               Field search module
  filehitcount          Count # of FileName hits form a user supplied file
  tcrr                  Perform temporal execution correlation on a user supplied filename
  ptcorr                Print temporal correlation context for the previously calculated tcrr
  tstomp                Attempt to detect modified last modification timestamps (experimental)
  tstack                Time stacking module (experimental)
  stack                 Good old stacking with a sql twist
  leven                 Find file name anomalies based on Levenshtein distance
  reconscan             Calculate recon activity in the database
  precon                Print contextual activity to recon commands identified on a host
  fevil                 Use temporal correlation on recon sessions to find potential evil (experimental)
  hashsearch            hashsearch module
  testset               Build fake testset database
```

3.3.3 Detection Techniques – Execution Tracing

You can read more about how this tool can be used to detect Temporal Execution Correlation, Time Stacking, etc., [here](#).

You can also download the tool from GitHub [here](#).



Memory Analysis



3.4 Memory Analysis

Traditional file-system detection techniques are highly unreliable when dealing with memory-resident malware, and therefore it is necessary to perform Memory analysis to detect malware, and also to understand what the purpose and capabilities of the malware are.



3.4 Memory Analysis

Memory forensics can provide unprecedented visibility into the run-time state of a system. It is possible to extract which processes were running, open network connections, and recently executed commands in a manner that is independent of the system. This will reduce the chance of sophisticated malware (rootkits for example) interfering with the results by, for example, modifying them. Moreover, it is likely that critical data exists in memory, such as encryption keys and memory-resident injected code fragments.



3.4 Memory Analysis

Malware often performs injection or system manipulation directly in memory to avoid detection. Some of the techniques often employed, which we'll discuss later are:

- Shellcode injection
- DLL and Reflective DLL injection
- Process hollowing
- API hooking
- Gargoyle

```
24 # @param experiment the experiment
25 # @param observations an array of Observations, or None
26 # @param control the control Observation
27
28 def initialize(experiment, observations = [], control = None):
29     @experiment = experiment
30     @observations = observations
31     @control = control
32     @candidates = observations + [control]
33     evaluate_candidates
34
35     freeze
36
37     # @param the experiment's context
38     def context:
39         experiment.context
40
41     # @param the name of the experiment
42     def experiment_name:
43         experiment.name
44
45     # @param the result a match between an observation and a control
46     def match:
47         ...
48
49     @science/result: 1.1
```



3.4 Memory Analysis

However, before an analyst can analyze the memory, it needs to be acquired (also referred to as dumped, meaning a 1:1 mapping of physical memory to a file, called a memory image) first.

There are two approaches for acquiring memory from a physical device:

- Hardware acquisition
- Software acquisition

```
21 def __init__(self, context):
22     """Initialize the context"""
23     self.context = context
24     self.experiment_name = None
25     self.observations = []
26     self.control = None
27
28 def initialize_experiment(self, observations = [], control = None):
29     """Initialize the experiment"""
30     self.experiment = Experiment()
31     self.observations = observations
32     self.control = control
33     self.candidates = []
34     self.evaluate_candidates()
35
36 def evaluate_candidates(self):
37     """Evaluate the candidates"""
38     # TODO: Implement the evaluation logic
39
40 def __str__(self):
41     """String representation of the object"""
42     return f"Experiment: {self.experiment_name}"
43
44 def __repr__(self):
45     """Repr representation of the object"""
46     return f"Experiment: {self.experiment_name}"
47
48 def __eq__(self, other):
49     """Equality comparison"""
50     return self.experiment_name == other.experiment_name
51
52 def __neq__(self, other):
53     """Not equality comparison"""
54     return not self.__eq__(other)
55
56 def __lt__(self, other):
57     """Less than comparison"""
58     return self.experiment_name < other.experiment_name
59
60 def __gt__(self, other):
61     """Greater than comparison"""
62     return self.experiment_name > other.experiment_name
63
64 def __le__(self, other):
65     """Less than or equal comparison"""
66     return self.experiment_name <= other.experiment_name
67
68 def __ge__(self, other):
69     """Greater than or equal comparison"""
70     return self.experiment_name >= other.experiment_name
```

3.4 Memory Analysis

Hardware acquisition has the advantage of being more resilient against rootkit modification. It communicates directly the memory controller with no communication to the OS, which you may not be able to trust in the case of a compromised system. Hardware acquisition requires a PCI card to be installed to perform the acquisition.



3.4 Memory Analysis

Software acquisition is used to acquire the object at [\\Device \\PhysicalMemory](#) (essentially the Windows memory manager's view of the system). A software tool maps that object and reads its content, which requires kernel mode access to read it. Among some of the requirements for a stable tool are: OS support, memory footprint, ability to capture reserved sections without crashing the system, and output file support.



3.4 Memory Analysis

A drawback of using a software solution is that it will always require process and kernel memory (for itself), as it needs to execute and will therefore overwrite possible evidence. Another drawback is that software solutions are vulnerable to the previously mentioned rootkit modification attacks.



3.4 Memory Analysis

Some of the non-commercial tools available are [FTK](#), [Imager](#), [Dumplt](#), and [MAGNET RAM Capture](#).

<https://accessdata.com/product-download/ftk-imager-version-4-2-0>

<https://my.comae.com/>

<https://www.magnetforensics.com/resources/magnet-ram-capture/>

3.4 Memory Analysis

Memory can always be acquired from virtual machines. Some of the VM vendors provide the physical memory file directly if the guest OS has been suspended, or in a snapshot (such as VMware in a .vmem file). Sometimes, additional user interaction is required to generate a memory image, often performed in debugging mode (VirtualBox). Furthermore, memory dumps can be created from a system crash file, hibernation file, and more.



3.4 Memory Analysis

Before jumping into analysis mode, we need to outline what it is that we flag as suspicious on a generic level.



3.4 Memory Analysis

When identifying anomalies in processes, we are interested in:

- Image name - Legitimate process? Spelled correctly?
- Full Path - Appropriate path for system executable? Running from a user or a temp directory?
- Parent process - Is the parent process what you would expect?
- Command line - Do the arguments make sense?
- Start time - Was the process started at boot?
- Security identifier - Do the security identifiers make sense?
Why would a system process use a user account SID?

3.4 Memory Analysis

When identifying anomalies in network activity, we are interested in:

- Any process communicating over port 80, 443, or 8080 that is not a web browser
- Any browser not communicating over port 80, 443, or 8080
- Connections to unexplained internal or external IP addresses



3.4 Memory Analysis

When identifying anomalies in network activity, we are interested in (CONTINUED):

- Web requests directly to an IP addresses rather than a domain name
- RDP connections (port 3389), especially if originating from odd IP addresses (e.g. a static IP address assigned to a printer)
- Why does this process have network capability?
- DNS requests for unusual domain names

3.4 Memory Analysis

Moreover, other anomalies are:

- Unlinked processes
- Loaded suspicious DLLs
- Unlinked network connections
- Unmapped memory pages with execute privileges (code injection)
- Hooked API functions
- Known bad heuristics and signatures (e.g. YARA signatures).

3.4 Memory Analysis

Memory analysis is performed through the use of tools specifically designed for that purpose. Within this section, we'll look at these tools that will aid us in it:

- [Mandiant's \(FireEye\) Redline](#)
- [Volatility](#)
- [Get-InjectedThreat.ps1](#)
- [Memdump](#)

<https://www.fireeye.com/services/freeware/redline.html>

<https://github.com/volatilityfoundation/volatility>

<https://gist.github.com/jaredcatkinson/23905d34537ce4b5b1818c3e6405c1d2>

<https://github.com/marcosd4h/memhunter>

```
21 # @param: context - the context of the experiment
22 # @param: experiment - the experiment this result is for
23 # @param: observations - an array of Observations, in ascending order
24 # @param: control - the control Observation
25
26 def initialize(experiment, observations = [], control = nil)
27   @experiment = experiment
28   @observations = observations
29   @control = control
30   @candidates = observations - [control]
31   evaluate_candidates
32
33   freeze
34 end
35
36 # @param: the experiment's context
37 def context
38   experiment.context
39 end
40
41 # @param: the name of the experiment
42 def experiment_name
43   experiment.name
44 end
45
46 # @param: the result a match between all candidates
47 def match
48   result
49 end
50
51 def result
52   result
53 end
```

3.4.1 Memory Analysis - Redline

Redline is FireEye's free endpoint security tool that provides host investigative capabilities to find signs of malicious activity through memory and file analysis.

You can download Redline [here](https://www.fireeye.com/services/freeware/redline.html).

3.4.1 Memory Analysis - Redline

With Redline, you can:

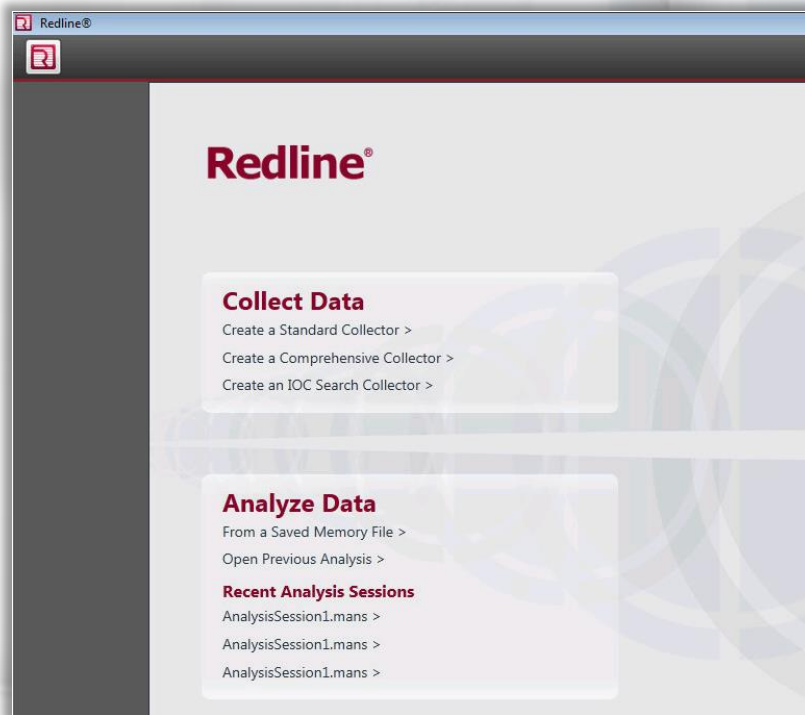
- Thoroughly audit and collect all running processes and drivers from memory, file-system metadata, registry data, event logs, network information, services, tasks and web history.
- Perform Indicators of Compromise (IOC) analysis. Supplied with a set of IOCs, the Redline Portable Agent is automatically configured to gather the data required to perform the IOC analysis and an IOC hit results review.

3.4.1 Memory Analysis - Redline

Redline is a GUI-based tool.

We can create portable agents that can gather a live memory capture of a computer system or many systems. We can also perform an IOC scan against the memory file.

We can also load a memory image and load saved Redline sessions (MANs files).



3.4.1 Memory Analysis - Redline

This tool automates the anomaly detection process and gives a quick overview of a particular machine's memory to detect rogue processes, injections, root kits, etc. using the MRI Score Index. Although not always accurate, Redline can still point you in the right direction with your analysis.

The next screen shot will show you the Redline interface along with an explanation as to what MRI Scores are.

3.4.1 Memory Analysis - Redline

Processes View

Home ▶ Host ▶ Processes ▶

Analysis Data

Filters

Enter string to find here... Reg Ex In All Fields Clear Column Filters Prev Next

	MRI	Process Name	MRI Score	PID	Path	Arguments
	57	csrss.exe	608	\??\C:\WINDOWS\system32	C:\WINDOWS\system32\csrss.exe Object	
	56	System	4			
	53	winlogon.exe	632	\??\C:\WINDOWS\system32	winlogon.exe	
	52	Explorer.EXE	1724	C:\WINDOWS	C:\WINDOWS\Explorer.EXE	
	51	TPAutoConnSvc.exe	1968	C:\Program Files\VMware\VMware Tools	"C:\Program Files\VMware\VMware Tools	
	51	TPAutoConnect.exe	1084	C:\Program Files\VMware\VMware Tools	TPAutoConnect.exe -q -i vmware -a COM	
	50	wuauclt.exe	1732	C:\WINDOWS\system32	"C:\WINDOWS\system32\wuauclt.exe" /R	
	50	wuauclt.exe	468	C:\WINDOWS\system32	"C:\WINDOWS\system32\wuauclt.exe"	
	50	svchost.exe	1028	C:\WINDOWS\System32	C:\WINDOWS\System32\svchost.exe -k n	
	50	VMwareUser.exe	452	C:\Program Files\VMware\VMware Tools	"C:\Program Files\VMware\VMware Tools	
	50	wscntfy.exe	888	C:\WINDOWS\system32	C:\WINDOWS\system32\wscntfy.exe	
	50	spoolsv.exe	1432	C:\WINDOWS\system32	C:\WINDOWS\system32\spoolsv.exe	

Review Processes by MRI Scores

MRI (Malware Risk Index) scoring uses a variety of techniques to assess the risk that a process is malware. Processes with a high MRI Score (up to 100) are more risky; those with a low score are less. Double click on a process name to view an MRI report that describes the reasons for that process's rating. MRI is intended as a guide for investigation; be aware that it can generate false positives and false negatives. These can be corrected in the MRI report.

All Processes ▶
Show all Processes.

[Redlined Processes](#)
Show only processes that have been determined to be of a high level of risk.

3.4.1 Memory Analysis - Redline

Hierarchical Processes View

Analysis Data | Home | Host | Hierarchical Processes

Filters

Enter string to find here... | Reg Ex | In All Fields | Clear Column Filters | Prev | Next

Processes
Hierarchical Processes
Driver Modules
Device Tree
Hooks
Timeline
Tags and Comments
Acquisition History

Review Processes Hierarchically
This view shows the relationship between all of the processes and their parent processes. It also displays the MRI scores for each of these processes and the processes which started them.

MRI (Malware Risk Index) scoring uses a variety of techniques to assess the risk that a process is malware. Processes with a high MRI Score (up to 100) are more risky; those with a low score are less. Double click on a process name to view an MRI report that describes the reasons for that process's rating. MRI is intended as a guide for investigation; be aware that it can generate false positives and false negatives. These can be corrected in the MRI report.

Process Name	MRI Score	PID	Path	Arguments	Username	Start Time	Kernel Ti...	User Time...	Hidden	Security...	SID T...	Parent Name	Par...
System	56	4				1601-01...	00:00:08	00:00:00		S-1-5-18			0
smss.exe	47	544	\SystemRoot\System32	\SystemRoot\System...		2010-08...	00:00:00	00:00:00		S-1-5-18	System		4
csrss.exe	57	608	\??\C:\WINDOWS\system32	C:\WINDOWS\sys...		2010-08...	00:00:00	00:00:00		S-1-5-18	smss.exe		544
winlogon.exe	53	632	\??\C:\WINDOWS\system32	winlogon.exe		2010-08...	00:00:01	00:00:00		S-1-5-18	smss.exe		544
services.exe	47	676	C:\WINDOWS\system32	C:\WINDOWS\sys...		2010-08...	00:00:02	00:00:00		S-1-5-18	winlogon.exe		632
alg.exe	48	216	C:\WINDOWS\System32	C:\WINDOWS\sys...		2010-08...	00:00:00	00:00:00		S-1-5-19	services.exe		676
vmacthlp.exe	48	844	C:\Program Files\VMware\VMware Tools	*C:\Program Files\...		2010-08...	00:00:00	00:00:00		S-1-5-18	services.exe		676
svchost.exe	47	856	C:\WINDOWS\system32	C:\WINDOWS\sys...		2010-08...	00:00:00	00:00:00		S-1-5-18	services.exe		676
svchost.exe	47	936	C:\WINDOWS\system32	C:\WINDOWS\sys...		2010-08...	00:00:00	00:00:00		S-1-5-20	services.exe		676
svchost.exe	50	1028	C:\WINDOWS\System32	C:\WINDOWS\sys...		2010-08...	00:00:03	00:00:00		S-1-5-18	services.exe		676
wuauclt.exe	50	468	C:\WINDOWS\system32	*C:\WINDOWS\sys...		2010-08...	00:00:00	00:00:00		S-1-5-2...	svchost.exe		1028

3.4.1 Memory Analysis - Redline

Processes > Handles

Home > Host > Processes > Handles

Analysis Data

Processes

- Handles
- Memory Sections
- Strings
- Ports
- Hierarchical Processes
- Driver Modules
- Device Tree
- Hooks
- Timeline
- Tags and Comments
- Acquisition History

Review Handles

The 'untrusted handles only' view filters the list of handles to eliminate those found in multiple trustworthy processes. The other filters allow you to view the different subcategories of Handles in isolation.

Show Named Handles

Display all named handles in the system.

[Show All Handles](#)

Enter string to find here...

Reg Ex In All Fields Clear Column Filters Prev Next

Trust Status	Handle Name	Handle Type	Occur...	Handle Index	Access Mask	Object Address
Untrusted	CritSecOutOfMemoryEvent	KeyedEvent	22	0x00000004	0x00000003	0xe1007e18
Untrusted	KnownDlls	Directory	22	0x00000008	0x00000003	0xe1533748
Untrusted	\Device\HarddiskVolume1\WINDOWS\system32	File	19	0x0000000c	0x00100020	0xf265cd8
Untrusted	\Device\HarddiskVolume1\WINDOWS\WinSxS\x86_Microsoft.Windows.Common-Controls_6595b64144cdf1df_6.0.2600.2180_x-ww_a84f1f9	File	65	0x00000010	0x00100020	0x80fcec0
Untrusted	Windows	Directory	22	0x00000014	0x0000000f	0xe1533d28
Untrusted	\Device\HarddiskVolume1\WINDOWS\WinSxS\x86_Microsoft.Windows.Common-Controls_6595b64144cdf1df_6.0.2600.2180_x-ww_a84f1f9	File	65	0x0000001c	0x00100020	0x80fb0ec0
Untrusted	BaseNamedObjects	Directory	23	0x00000028	0x0002000f	0xe1571708
Untrusted	SHIMUB_LOG_MUTEX	Mutant	6	0x0000002c	0x001f0001	0xf257148

3.4.1 Memory Analysis - Redline

Processes > Memory Sections

The screenshot displays the 'Memory Sections' view in the Redline application. The breadcrumb path at the top is 'Home > Host > Processes > Memory Sections'. The left sidebar, under 'Analysis Data', lists various system components, with 'Memory Sections' highlighted. The main area is titled 'Review Memory Sections / DLLs' and contains explanatory text about memory sections and DLLs. To the right, a table lists the loaded DLLs for several processes.

Analysis Data

- Processes
- Handles
- Memory Sections**
- Strings
- Ports
- Hierarchical Processes
- Driver Modules
- Device Tree
- Hooks
- Timeline
- Tags and Comments
- Acquisition History

Filters

Review Memory Sections / DLLs

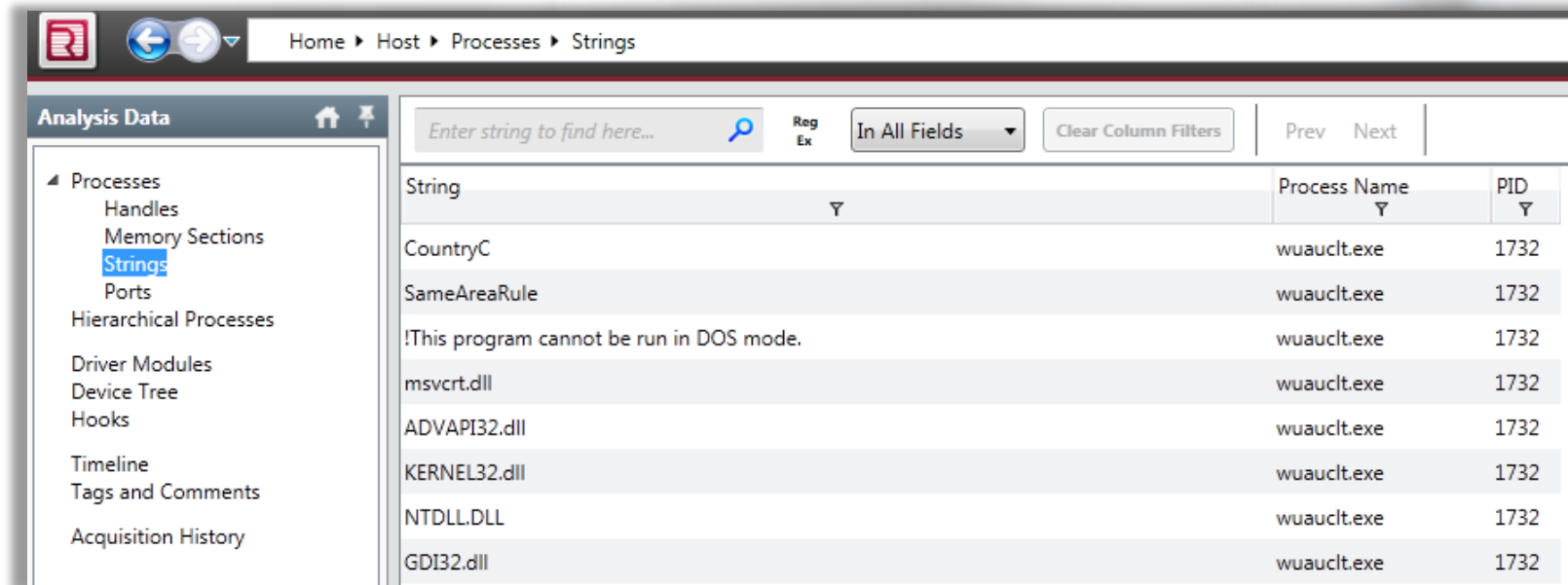
These views show the memory sections that each running process is comprised of. Named memory sections are those that are mapped to files, primarily DLLs. For those unfamiliar with malware analysis, the best view to start with is "Least Frequency of Occurrence (Untrusted Only)": unlike system DLLs, malware DLLs normally are not signed and are usually loaded by a single process, and thus will often appear in this view.

Enter string to find here... In All Fields

Trust Status	PID	ProcessName	Injected	Protection
Injected	1732	wuauclt.exe	✓	EXECUTE_READWRITE...
Injected	468	wuauclt.exe	✓	EXECUTE_READWRITE...
Injected	1028	svchost.exe	✓	EXECUTE_READWRITE...
Injected	856	svchost.exe	✓	EXECUTE_READWRITE...
Injected	4	System	✓	EXECUTE_READWRITE...
Injected	4	System	✓	EXECUTE_READWRITE...
Injected	4	System	✓	EXECUTE_READWRITE...
Injected	1968	TPAutoConnSvc.exe	✓	EXECUTE_READWRITE...

3.4.1 Memory Analysis - Redline

Processes > Strings

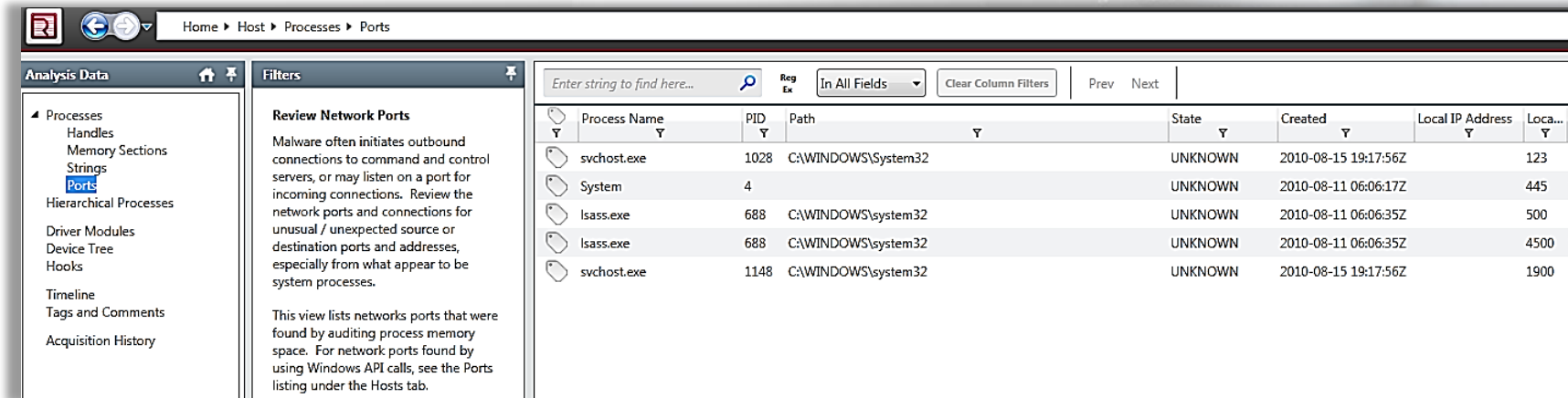


The screenshot displays the Redline application's 'Processes > Strings' view. The breadcrumb trail at the top reads 'Home > Host > Processes > Strings'. Below the breadcrumb, there is a search bar with the placeholder text 'Enter string to find here...' and a magnifying glass icon. To the right of the search bar are buttons for 'Reg Ex', 'In All Fields', 'Clear Column Filters', and 'Prev Next'. The main content area is a table with three columns: 'String', 'Process Name', and 'PID'. The table lists several strings found in the process 'wuauclt.exe'.

String	Process Name	PID
CountryC	wuauclt.exe	1732
SameAreaRule	wuauclt.exe	1732
!This program cannot be run in DOS mode.	wuauclt.exe	1732
msvcrt.dll	wuauclt.exe	1732
ADVAPI32.dll	wuauclt.exe	1732
KERNEL32.dll	wuauclt.exe	1732
NTDLL.DLL	wuauclt.exe	1732
GDI32.dll	wuauclt.exe	1732

3.4.1 Memory Analysis - Redline

Processes > Ports



Home > Host > Processes > Ports

Analysis Data

- Processes
 - Handles
 - Memory Sections
 - Strings
 - Ports**
- Hierarchical Processes
- Driver Modules
- Device Tree
- Hooks
- Timeline
- Tags and Comments
- Acquisition History

Filters

Review Network Ports

Malware often initiates outbound connections to command and control servers, or may listen on a port for incoming connections. Review the network ports and connections for unusual / unexpected source or destination ports and addresses, especially from what appear to be system processes.

This view lists network ports that were found by auditing process memory space. For network ports found by using Windows API calls, see the Ports listing under the Hosts tab.

Process Name	PID	Path	State	Created	Local IP Address	Local Port
svchost.exe	1028	C:\WINDOWS\System32	UNKNOWN	2010-08-15 19:17:56Z		123
System	4		UNKNOWN	2010-08-11 06:06:17Z		445
lsass.exe	688	C:\WINDOWS\system32	UNKNOWN	2010-08-11 06:06:35Z		500
lsass.exe	688	C:\WINDOWS\system32	UNKNOWN	2010-08-11 06:06:35Z		4500
svchost.exe	1148	C:\WINDOWS\system32	UNKNOWN	2010-08-15 19:17:56Z		1900

3.4.1 Memory Analysis - Redline

Again, Redline is good to get a quick look at a machine's memory. This process is known as triaging. When you triage, you're getting a 30,000 foot view of what is going on.

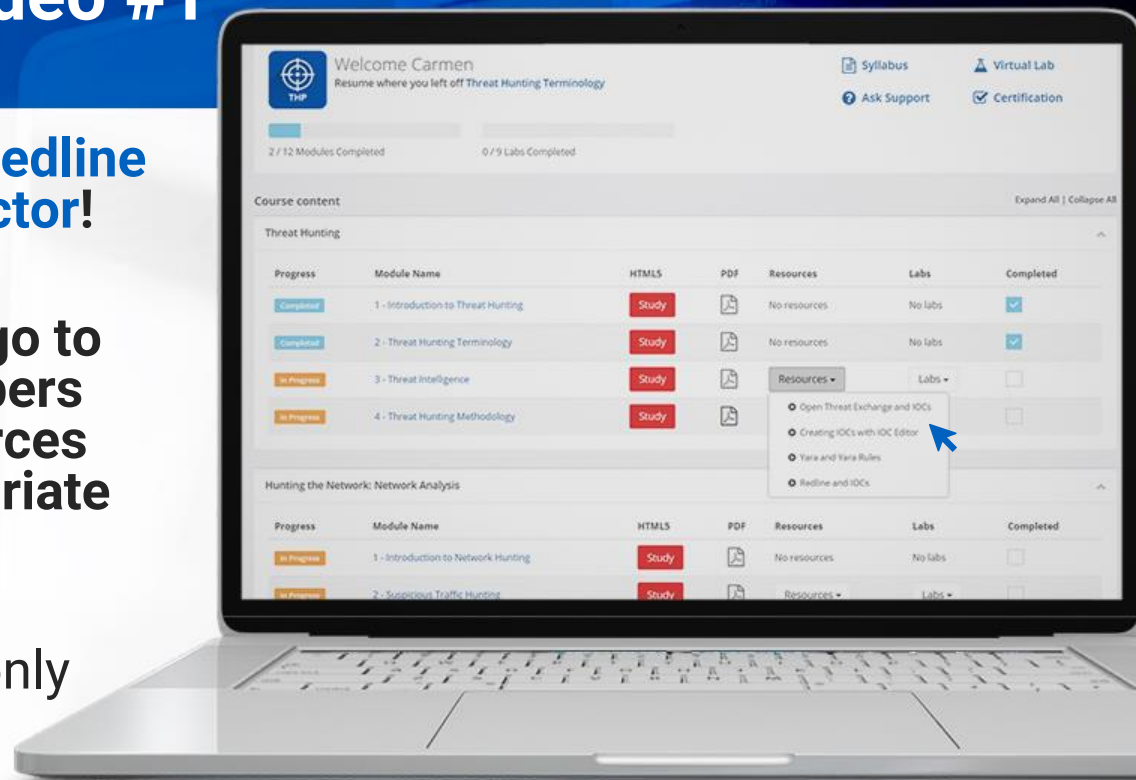
If something is detected as malicious by Redline, then you can take a closer examination with a more advanced tool, such as Volatility.

3.4.1.1 Redline Video #1

Check out the video on **Redline** – **Create Standard Collector!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).

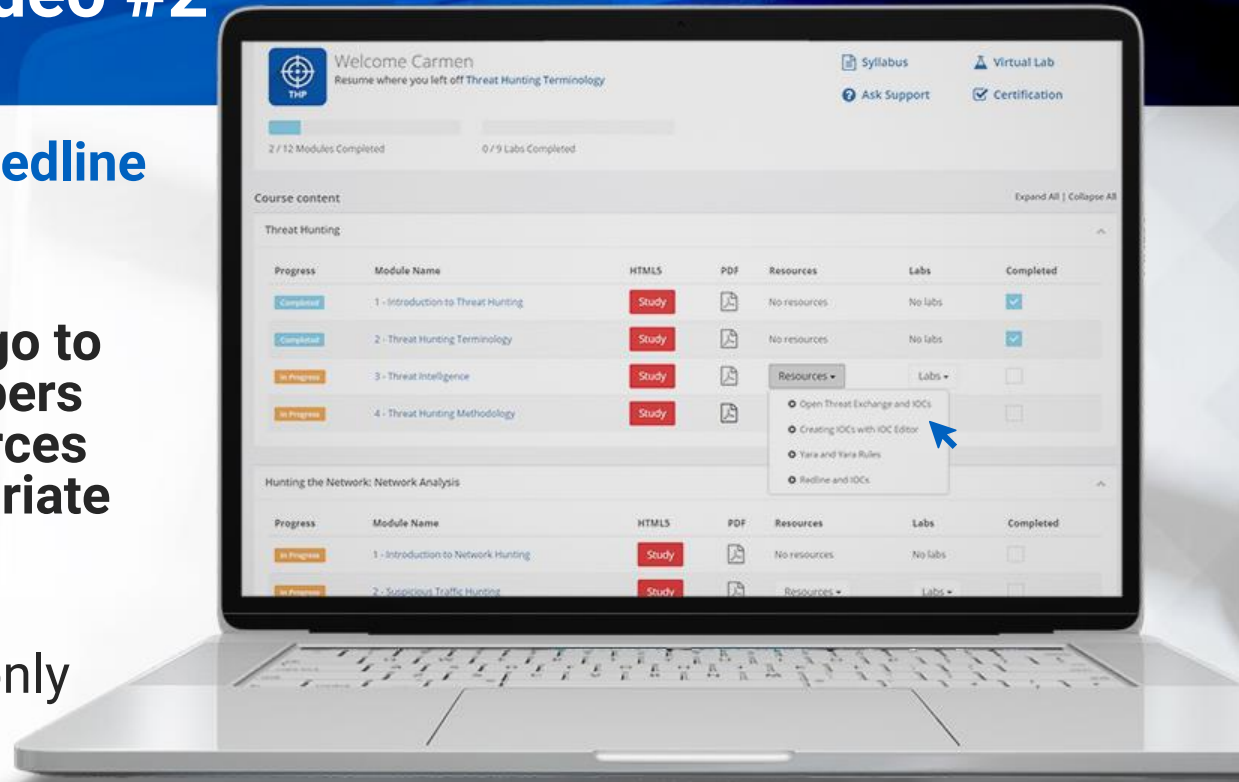


3.4.1.2 Redline Video #2

Check out the video on **Redline – Basic Usage!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).

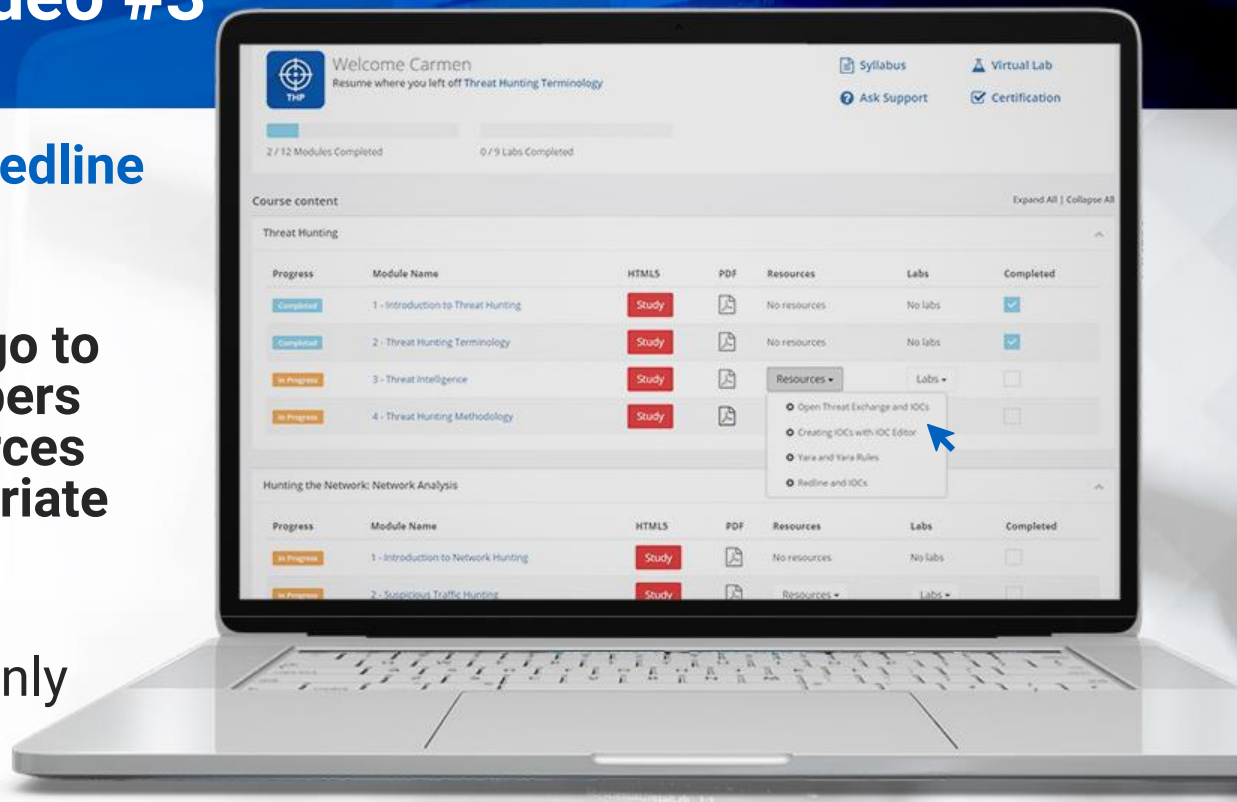


3.4.1.3 Redline Video #3

Check out the video on **Redline – Create Analysis File!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).

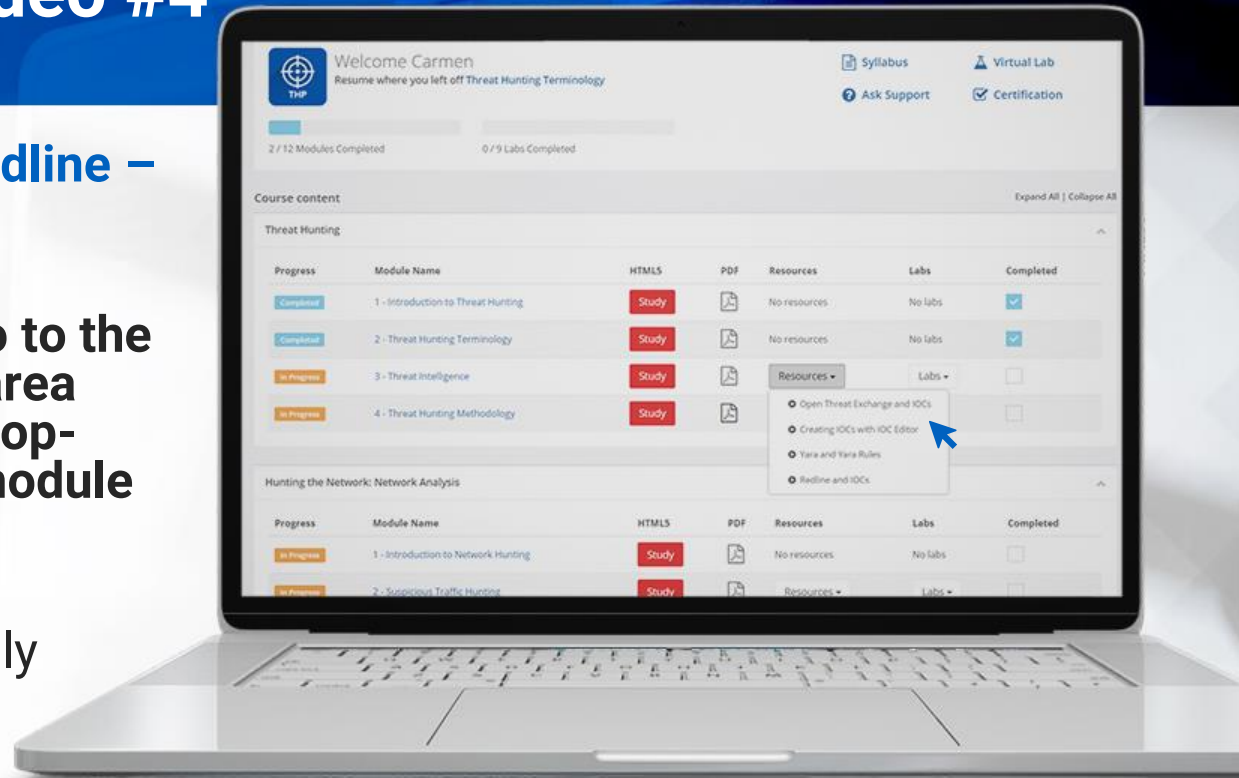


3.4.1.4 Redline Video #4

Check out the video on **Redline – Detecting Code Injection!**

To **ACCESS** your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course.
To upgrade, click [LINK](#).



3.4.2 Memory Analysis - Volatility

“The Volatility Framework is a completely open collection of tools, implemented in Python under the GNU General Public License, for the extraction of digital artifacts from volatile memory (RAM) samples. The extraction techniques are performed completely independent of the system being investigated but offer visibility into the runtime state of the system.” – [Volatility on GitHub](https://github.com/volatilityfoundation/volatility)

3.4.2 Memory Analysis - Volatility

Volatility is not as user friendly as Redline, but is definitely an excellent tool that is worth learning and getting comfortable with. Volatility will be able to detect malicious activity that Redline might miss.



3.4.2 Memory Analysis - Volatility

Volatility is available for Windows, Linux, and Mac OS and is written purely in Python. In order to perform an analysis in Volatility, we need to specify three parameters:

- Memory dump file
- OS Profile
- Plugin (also called Module)

```
34 def __init__(self, observations = [], control = null)
35     self.observations = observations
36     self.control = control
37     self.candidates = observations + [control]
38     self.evaluate_candidates()
39
40     def freeze(self):
41         """
42         Freeze the experiment's context
43         """
44         self.context = experiment.context
45         self.freeze()
46
47     def __str__(self):
48         """
49         Return the name of the experiment
50         """
51         return self.experiment_name
52
53     def __repr__(self):
54         """
55         Return the result a match between all
56         """
57         return self.matcher
58
59     def __getitem__(self, item):
60         return self.matcher[item]
```

3.4.2 Memory Analysis - Volatility

An Operating System profile is required because each version of an Operating System has different definition and implementation of memory objects, so this tells Volatility how to treat the memory image in order to find data structures in it.

By default, Volatility comes with all existing Windows profiles from Windows XP to Windows 10.

3.4.2 Memory Analysis - Volatility

The plugin is the payload of the command. It tells Volatility what we are looking for in the memory image.

Currently, Volatility supports over 200 plugins by default, and the analyst has the opportunity to extend Volatility's capabilities by developing custom plugins. Some of the plugins are shown on the next slide.

3.4.2 Memory Analysis - Volatility

Supported Plugin Commands:

amcache	Print AmCache information
apihooks	Detect API hooks in process and kernel memory
atoms	Print session and window station atom tables
atomscan	Pool scanner for atom tables
auditpol	Prints out the Audit Policies from HKLM\SECURITY\Policy\PolAdeEv
bigpools	Dump the big page pools using BigPagePoolScanner
bioskbd	Reads the keyboard buffer from Real Mode memory
cachedump	Dumps cached domain hashes from memory
callbacks	Print system-wide notification routines
clipboard	Extract the contents of the windows clipboard
cmdline	Display process command-line arguments
cmdscan	Extract command history by scanning for _COMMAND_HISTORY
connections	Print list of open connections [Windows XP and 2003 Only]
connscan	Pool scanner for tcp connections
consoles	Extract command history by scanning for _CONSOLE_INFORMATION
crashinfo	Dump crash-dump information
deskscan	Poolscanner for tagDESKTOP (desktops)
devicetree	Show device tree
dlldump	Dump DLLs from a process address space
dlllist	Print list of loaded dlls for each process
driverirp	Driver IRP hook detection
drivermodule	Associate driver objects to kernel modules
driverscan	Pool scanner for driver objects
dumpcerts	Dump RSA private and public SSL keys
dumpfiles	Extract memory mapped and cached files
dumpregistry	Dumps registry files out to disk
editbox	Displays information about Edit controls. (Listbox experimental.)
envvars	Display process environment variables
eventhooks	Print details on windows event hooks

3.4.2 Memory Analysis - Volatility

As mentioned, before starting the analysis, Volatility requires the OS version to be specified as a command line argument.

Often times, as an analyst, you would know that, but in the cases you don't, a helpful plugin is "imageinfo", which identifies (to its best capabilities) the OS version from the memory dump itself as shown on the next slide.

3.4.2 Memory Analysis - Volatility

```
root@THP: /mnt/hgfs/shared
root@THP:/mnt/hgfs/shared# volatility -f W10.vmem imageinfo
Volatility Foundation Volatility Framework 2.6
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : Win10x64_17134, Win10x64_14393, Win10x64_10586,
Win10x64_16299, Win2016x64_14393, Win10x64_15063 (Instantiated with Win10x64_15
063)
      AS Layer1 : SkipDuplicatesAMD64PagedMemory (Kernel AS)
      AS Layer2 : VMWareMetaAddressSpace (Unnamed AS)
      AS Layer3 : FileAddressSpace (/mnt/hgfs/shared/W10.vmem)
      PAE type : No PAE
      DTB : 0xlad002L
      KDBG : 0xf80031c49520L
      Number of Processors : 2
      Image Type (Service Pack) : 0
      KPCR for CPU 0 : 0xffffffff80030b12000L
      KPCR for CPU 1 : 0xffff8a01a01cf000L
      KUSER_SHARED_DATA : 0xffffffff780000000000L
      Image date and time : 2019-10-25 00:56:46 UTC+0000
      Image local date and time : 2019-10-25 02:56:46 +0200
root@THP:/mnt/hgfs/shared#
```

3.4.2 Memory Analysis - Volatility

When the plugin finishes executing, Volatility presents us back with a list of potential OS profiles, sorted by the most likely one.

In this example, the profile is "Win10x64_17134". Armed with the profile, we can continue and begin the analysis.

3.4.2 Memory Analysis - Volatility

One of the basic functions of Volatility is to list processes running on the system with the plugin "pslist". In order to locate processes with "pslist", Volatility is locating the doubly-linked list that keeps track of the processes in memory, and displays them back to the user. This is the equivalent of the processes list in task manager on a running Windows system

3.4.2 Memory Analysis - Volatility

Note that the output may include information on processes that have already terminated, which includes their exit time. This can be particularly useful in cases where a process, such as cmd.exe, is used to start a malicious executable and exits afterwards.

An example of this plugin is shown on the next slide.

3.4.2 Memory Analysis - Volatility

```
root@THP:/mnt/hgfs/shared# volatility -f W10.vmem --profile Win10x64_17134 pslist
```

Volatility Foundation Volatility Framework 2.6

Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exit
0xffffe48904ac5040	System	4	0	96	0	-----	0	2019-10-24 04:13:11 UTC+0000	
0xffffe48904b51040	Registry	88	4	3	0	-----	0	2019-10-24 04:13:02 UTC+0000	
0xffffe48905e92040	smss.exe	280	4	2	0	-----	0	2019-10-24 04:13:11 UTC+0000	
0xffffe48905e34580	csrss.exe	384	376	10	0	0	0	2019-10-24 04:13:12 UTC+0000	
0xffffe489067cc080	wininit.exe	468	376	1	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe489063e7080	csrss.exe	476	460	12	0	1	0	2019-10-24 04:13:13 UTC+0000	
0xffffe489067ef080	winlogon.exe	536	460	6	0	1	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48905293080	services.exe	604	468	9	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48905384080	lsass.exe	620	468	8	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48906886580	svchost.exe	724	604	2	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48906882580	fontdrvhost.exe	748	468	5	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48906884580	fontdrvhost.exe	756	536	5	0	1	0	2019-10-24 04:13:13 UTC+0000	
0xffffe4890684b080	svchost.exe	764	604	18	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48906870580	svchost.exe	864	604	20	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48906848580	svchost.exe	908	604	6	0	0	0	2019-10-24 04:13:13 UTC+0000	
0xffffe48906ac8080	dwm.exe	968	536	12	0	1	0	2019-10-24 04:13:14 UTC+0000	
0xffffe48906ac6400	LogonUI.exe	976	536	0	-----	1	0	2019-10-24 04:13:14 UTC+0000	2019-10-25 00:46:45 UTC+0000
0xffffe48906b77580	svchost.exe	600	604	16	0	0	0	2019-10-24 04:13:14 UTC+0000	
0xffffe48906b75580	svchost.exe	796	604	4	0	0	0	2019-10-24 04:13:14 UTC+0000	

3.4.2 Memory Analysis - Volatility

Malware, specifically rootkits, often tries to hide its existence by unlinking itself from this list (amongst other techniques), in which case the process will not be shown in the output by pslist.

Fortunately, in memory, we can locate processes by other means, such as searching through the memory dump and finding data structures that match that of an "_EPROCESS", the representative structure of a process in memory. By doing so, we can identify even hidden processes. For this purpose, we have at our disposal the plugin "psscan".

3.4.2 Memory Analysis - Volatility

"Psscan" scans the entire memory dump and reports on any identified objects that have the structure of an `_EPROCESS`.

In some cases, this plugin may return false positives, and also processes that have finished execution some time ago (in some occasions, even from a previous reboot).

3.4.2 Memory Analysis - Volatility

Yet another, and even more powerful plugin that we may utilize to identify hidden processes, is "psxview", which uses multiple techniques for finding processes in memory. It then reports the output in a single view by displaying whether or not a certain process exists for each of the detection techniques.

The next slide shows the output of "psxview".

3.4.2 Memory Analysis - Volatility

Note that “THP.exe” was hidden from “pslist” among others.

```
root@THP:/mnt/hgfs/shared# volatility -f THP.vmem --profile=Win10x86_14393 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)  Name                PID pslist psscan thrddproc pspcid csrss session deskthrd ExitTime
-----
0x00870b00 services.exe        588 True   True   True   False True True   False
0x53a08380 svchost.exe         712 True   True   True   False True True   False
0x3e85ec40 ShellExperienc 1772 True   True   True   False True True   False
0x6da37040 dlhlosh.exe       2448 True   True   True   False True True   False
0x667a8040 svchost.exe       1796 True   True   True   False True True   False
0x57eef140 svchost.exe       368 True   True   True   False True True   False
0x4c2e6c40 SystemSettings 5384 True   True   True   False True True   False
0x5fabcl80 svchost.exe       2028 True   True   True   False True True   False
0x0ca7b040 svchost.exe       2296 True   True   True   False True True   False
0x030b0240 vmtoolsd.exe     516 True   True   True   False True True   False
0x5fba1280 vmtoolsd.exe     2036 True   True   True   False True True   False
0x4ea612c0 conhost.exe        4416 True   True   True   False True True   False
0x5b8fd040 dlhlosh.exe       5248 True   True   True   False True True   False
0x53c37040 dwm.exe         836 True   True   True   False True True   False
0x7ee673c0 svchost.exe       1236 True   True   True   False True True   False
0x52f1f040 svchost.exe       984 True   True   True   False True True   False
0x89391400 GoogleUpdate.e 3344 True   True   True   False True True   False
0x5e716ac0 svchost.exe       1080 True   True   True   False True True   False
0x6e262440 msdtc.exe           2556 True   True   True   False True True   False
0x7eae0040 vmacthlp.exe     1276 True   True   True   False True True   False
0x11e97480 taskshostv.exe 3312 True   True   True   False True True   False
0x6fa28c40 powershell.exe 4284 True   True   True   False True True   False
0x6738f540 Calculator.exe      5148 True   True   True   False True True   False
0x5de2f540 lsass.exe        596 True   True   True   False True True   False
0x4ea616c0 cmd.exe         4408 True   True   True   False True True   False
0x63c655c0 svchost.exe       1548 True   True   True   False True True   False
0x54563040 svchost.exe       668 True   True   True   False True True   False
0x032b5600 svchost.exe       3116 True   True   True   False True True   False
0x5f40f640 ApplicationFra 5176 True   True   True   False True True   False
0x647f0680 vmtoolsd.exe     3472 True   True   True   False True True   False
0x099816c0 conhost.exe       4960 True   True   True   False True True   False
0x5d17a040 wininit.exe        464 True   True   True   False True True   False
0x7efb3780 WmiPrvSE.exe 2332 True   True   True   False True True   False
0x52f09840 svchost.exe       944 True   True   True   False True True   False
0x3c1bd040 cmd.exe         5784 True   True   True   False True True   False
0x6d499880 RuntimeBroker. 2880 True   True   True   False True True   False
0x5fc30980 VgAuthService. 2044 True   True   True   False True True   False
0x65e41940 Service.exe        1952 True   True   True   False True True   False
0x6a56b040 SearchIndexer. 3552 True   True   True   False True True   False
0x11abb040 slhosh.exe       2684 True   True   True   False True True   False
0x67932ac0 spoolsv.exe   1788 True   True   True   False True True   False
0x5e4bb880 svchost.exe       960 True   True   True   False True True   False
0x43effbc0 MSOSYNC.EXE 3488 True   True   True   False True True   False
0x4accf040 KeePass.exe      3952 True   True   True   False True True   False
0x7eefcf00 svchost.exe       1448 True   True   True   False True True   False
0x45a35780 conhost.exe     5792 True   True   True   False True True   False
0x4cf8dc40 THP.exe            4944 False  True   True   False True True   False
0x00000000 svchost.exe       1920 True   True   True   False True True   False
```

3.4.2 Memory Analysis - Volatility

In the hunt for malicious processes, often times we attempt to identify anomalies, such as whether a process has been started by an expected Parent Process. For this purpose, we can utilize "pstree" in Volatility, whose output is a dot-aligned listing as shown on the next slide. With this view, we can identify obvious anomalies, such as if the parent process of svchost.exe is not services.exe. Another example would be if notepad.exe is starting PowerShell.

Note that this plugin will not include hidden processes in its output!

3.4.2 Memory Analysis - Volatility

Output of “pstree”:

```
root@THP:/mnt/hgfs/shared# volatility -f THP.vmem --profile=Win10x86_14393 pstree
```

```
Volatility Foundation Volatility Framework 2.6
```

Name	Pid	PPid	Thds	Hnds	Time
0x9308c040:wininit.exe	464	352	2	0	2019-10-26 17:08:35 UTC+0000
. 0x8b787b00:services.exe	588	464	5	0	2019-10-26 17:08:36 UTC+0000
.. 0x9536d040:svchost.exe	1920	588	12	0	2019-10-26 17:08:45 UTC+0000
.. 0x9532b040:svchost.exe	1032	588	15	0	2019-10-26 17:08:40 UTC+0000
.. 0xa6179600:svchost.exe	3116	588	7	0	2019-10-26 17:10:26 UTC+0000
.. 0x9526b5c0:svchost.exe	1548	588	9	0	2019-10-26 17:08:44 UTC+0000
.. 0xa6029040:dllhost.exe	2448	588	11	0	2019-10-26 17:08:50 UTC+0000
.. 0x93124040:svchost.exe	668	588	23	0	2019-10-26 17:08:37 UTC+0000
... 0x8ff0f880:RuntimeBroker.	2880	668	34	0	2019-10-26 17:20:55 UTC+0000
... 0x8b102c40:SystemSettings	5384	668	19	0	2019-10-26 17:30:18 UTC+0000
... 0x9f6cec40:SearchUI.exe	2520	668	44	0	2019-10-26 17:20:56 UTC+0000

3.4.2 Memory Analysis - Volatility

Volatility's "netscan" plugin traverses memory and identifies all memory structures that represent a network connection. Similar to "psscan", you may find false positives in its output. However, it may also display connections which are no longer active that are still preserved in memory.

The next thing that we'll look at is code injection. At this point, we assume that the reader is familiar with the basic structure of a Portable Executable (PE) file. If not, you can refer to this [link](https://resources.infosecinstitute.com/2-malware-researchers-handbook-demystifying-pe-file/#gref) and read more about it.

3.4.2 Memory Analysis - Volatility

In general, executable code resides in the ".text" section of a PE file, both when it's located on disk and also when loaded into memory. With a few exceptions, this is where executable code should reside.

An Injected code will not show in the text section, as it will be placed on the "heap" of a process. Let's look at the concept of DLL Injection.

```
25 def initialize(experiment, observations = {}, control = null)
26   @experiment = experiment
27   @observations = observations
28   @control = control
29   @candidates = observations.keys
30   evaluate_candidates
31
32   freeze
33 end
34
35 # Public: The agent's control
36 def control
37   @control
38 end
39
40 # Public: The result of the experiment
41 def result
42   @result
43 end
44
45 # Public: Was the result a match between the
46 def matched?
47   @matched
48 end
49
50 # Private: result.rb 1.1
```

3.4.2 Memory Analysis - Volatility

DLL Injection is the process of inserting code into a running process. The code inserted is in the form of a Dynamic Link Library (DLL), mainly because DLLs are meant to be loaded as needed at run time. Although, this does not mean that injection of other types of assembly is not possible, such as executables or simply handwritten shellcode.



3.4.2 Memory Analysis - Volatility

Injecting into SYSTEM process or process from another context (eg. process of another user) requires certain privileges (more specifically, SeDebugPrivilege, which is required to debug and adjust the memory of another process).

This is usually achieved through administrative rights on the machine.



3.4.2 Memory Analysis - Volatility

Although there are multiple varieties of code injection techniques, the most generic one is a 4-step process where the Win32 API is used to provide the necessary functionality. The steps are:

1. Attach to the victim process
2. Allocate memory within the victim process
3. Copy the DLL or the DLL Path into the allocated memory
4. Instruct the process to Execute the DLL

3.4.2 Memory Analysis - Volatility

Each of these steps is associated with one or more Win32 API function calls.

During step 3, the malware author has the option to either inject the path of the DLL on disk and load then execute it, or inject the DLL itself, if the allocated memory in the victim process is large enough.

```
20 def __init__(self, path, name, control):
21     self.path = path
22     self.name = name
23     self.control = control
24     self.experiment = experiment
25     self.observations = observations
26     self.control = control
27
28 def initialize(self, experiment, observations = [], control = None):
29     @experiment = experiment
30     @observations = observations
31     @control = control
32
33     freeze
34     context
35     experiment.context
36     experiment.name
37     experiment.name
38
39     # Check the results a match between the
40     def match
41     for result in result:
42         if result == 1:
```



3.4.2 Memory Analysis - Volatility

A drawback in the steps performed during the injection for malware authors, is that the DLL that is being injected may need to be located on disk and could potentially be caught by Antivirus software. Through static analysis, it may be possible to identify injection capability by just observing the import headers of a PE file (if it is not obfuscated in some way). Unlike a Simple DLL Injection, the power of Reflective DLL Injection comes in that it is able to inject to and execute directly from memory.



3.4.2 Memory Analysis - Volatility

Reflective Injection is a special technique of code injection, where code is injected and loaded from memory, directly in the process itself. This type of injection is often used to further expand the capabilities of a functionality limited stager, by delivering additional modules only when needed. The library loading itself is not registered in any way with the host system, and as a result, it is largely undetectable at both a system and process level.



3.4.2 Memory Analysis - Volatility

Techniques of detecting code injections have been around for a while.

One of the most famous is scanning through private memory regions (the heap of a process) and identifying those that have the executable bit set (RWE or RX), and/or have no memory mapped file present on disk (unmapped binary file).

3.4.2 Memory Analysis - Volatility

An unmapped process binary is an indication of process hollowing.

A detailed explanation and research on process hollowing is available [here](#).

```

23 # the experiment will result in one
24 # observations: an array of observations, an experiment
25 # control: the control observation
26
27 def initialize(experiment, observations = [], control = nil)
28   @experiment = experiment
29   @observations = observations
30   @control = control
31   @candidates = observations - [control]
32   evaluate_candidates
33
34   freeze
35 end
36
37 def context
38   experiment.context
39 end
40
41 # Public: the name of the experiment
42 def experiment_name
43   @experiment_name
44 end
45
46 # Public: the result of a search between all candidates
47 def matched?
48
49   IO.puts "result: 1, 1"

```

3.4.2 Memory Analysis - Volatility

The detective techniques mentioned so far are employed by Volatility's malfind plugin for detecting code injection.

Among other details, in malfind's output we can see:

- Process name and PID where injection was detected
- Offset address of where the injection was detected
- Hex, ASCII, and Disassembly view of the injected area

3.4.2 Memory Analysis - Volatility

Injected areas that begin with the "MZ" file header are especially interesting to us – denoting a Windows executable file (which is the case on the picture on the next slide). Of course, the injected area may contain shellcode, which lacks the “MZ” header, and which requires that the analyst further investigate to understand its behavior and purpose.

Note: malfind's output may contain false positives.

3.4.2 Memory Analysis - Volatility

Partial output of “malfind” – “MZ” header detected.

```
Process: svchost.exe Pid: 1148 Address: 0x9f0000
VadTag: Vads Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 38, MemCommit: 1, PrivateMemory: 1, Protection: 6

0x009f0000 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00 MZ.....
0x009f0010 b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 .....@.....
0x009f0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0x009f0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
tools
0x009f0000 4d          DEC EBP
0x009f0001 5a          POP EDX
0x009f0002 90          NOP
0x009f0003 0003        ADD [EBX], AL
0x009f0005 0000        ADD [EAX], AL
```

3.4.2 Memory Analysis - Volatility

The following [resource](#) contains additional descriptions of other injection techniques and their respective detection.

3.4.2 Memory Analysis - Volatility

Volatility has a plugin "yarascan" which allows you to search for strings, patterns, and also compound rules. As stated on its [wiki page](#), this plugin can help you locate any sequence of bytes (like assembly instructions with wild cards), regular expressions, ANSI strings, or Unicode strings in user mode or kernel memory.

You can also use a YARA rules file as an argument instead of specifying the rule(s) on the command line.

3.4.2 Memory Analysis - Volatility

In some instances, you may be hunting for very sophisticated pieces of malware (rootkits) where you have to dig into system objects such as drivers, mutexes, and hooked functions.



3.4.2 Memory Analysis - Volatility

The following plugins are extremely helpful in this area:

- idt
- ssdt
- apihooks
- modules
- modscan
- driverirp
- driverscan

An example and walkthrough of rootkit detection is available [here](#).

```
20 def __init__(self):
21     self._context = None
22
23     # The experiment's context
24     @property
25     def context(self):
26         return self._context
27
28     @context.setter
29     def context(self, context):
30         self._context = context
31
32     # The experiment's observations
33     @property
34     def observations(self):
35         return self._observations
36
37     @observations.setter
38     def observations(self, observations):
39         self._observations = observations
40
41     # The control observation
42     @property
43     def control(self):
44         return self._control
45
46     @control.setter
47     def control(self, control):
48         self._control = control
49
50     # The candidates
51     @property
52     def candidates(self):
53         return self._candidates
54
55     @candidates.setter
56     def candidates(self, candidates):
57         self._candidates = candidates
58
59     # The results
60     @property
61     def results(self):
62         return self._results
63
64     @results.setter
65     def results(self, results):
66         self._results = results
67
68     # The matchers
69     @property
70     def matchers(self):
71         return self._matchers
72
73     @matchers.setter
74     def matchers(self, matchers):
75         self._matchers = matchers
76
77     # The results
78     @property
79     def results(self):
80         return self._results
81
82     @results.setter
83     def results(self, results):
84         self._results = results
85
86     # The matchers
87     @property
88     def matchers(self):
89         return self._matchers
90
91     @matchers.setter
92     def matchers(self, matchers):
93         self._matchers = matchers
94
95     # The results
96     @property
97     def results(self):
98         return self._results
99
100    @results.setter
101    def results(self, results):
102        self._results = results
103
104    # The matchers
105    @property
106    def matchers(self):
107        return self._matchers
108
109    @matchers.setter
110    def matchers(self, matchers):
111        self._matchers = matchers
112
113    # The results
114    @property
115    def results(self):
116        return self._results
117
118    @results.setter
119    def results(self, results):
120        self._results = results
121
122    # The matchers
123    @property
124    def matchers(self):
125        return self._matchers
126
127    @matchers.setter
128    def matchers(self, matchers):
129        self._matchers = matchers
130
131    # The results
132    @property
133    def results(self):
134        return self._results
135
136    @results.setter
137    def results(self, results):
138        self._results = results
139
140    # The matchers
141    @property
142    def matchers(self):
143        return self._matchers
144
145    @matchers.setter
146    def matchers(self, matchers):
147        self._matchers = matchers
148
149    # The results
150    @property
151    def results(self):
152        return self._results
153
154    @results.setter
155    def results(self, results):
156        self._results = results
157
158    # The matchers
159    @property
160    def matchers(self):
161        return self._matchers
162
163    @matchers.setter
164    def matchers(self, matchers):
165        self._matchers = matchers
166
167    # The results
168    @property
169    def results(self):
170        return self._results
171
172    @results.setter
173    def results(self, results):
174        self._results = results
175
176    # The matchers
177    @property
178    def matchers(self):
179        return self._matchers
180
181    @matchers.setter
182    def matchers(self, matchers):
183        self._matchers = matchers
184
185    # The results
186    @property
187    def results(self):
188        return self._results
189
190    @results.setter
191    def results(self, results):
192        self._results = results
193
194    # The matchers
195    @property
196    def matchers(self):
197        return self._matchers
198
199    @matchers.setter
200    def matchers(self, matchers):
201        self._matchers = matchers
202
203    # The results
204    @property
205    def results(self):
206        return self._results
207
208    @results.setter
209    def results(self, results):
210        self._results = results
211
212    # The matchers
213    @property
214    def matchers(self):
215        return self._matchers
216
217    @matchers.setter
218    def matchers(self, matchers):
219        self._matchers = matchers
220
221    # The results
222    @property
223    def results(self):
224        return self._results
225
226    @results.setter
227    def results(self, results):
228        self._results = results
229
230    # The matchers
231    @property
232    def matchers(self):
233        return self._matchers
234
235    @matchers.setter
236    def matchers(self, matchers):
237        self._matchers = matchers
238
239    # The results
240    @property
241    def results(self):
242        return self._results
243
244    @results.setter
245    def results(self, results):
246        self._results = results
247
248    # The matchers
249    @property
250    def matchers(self):
251        return self._matchers
252
253    @matchers.setter
254    def matchers(self, matchers):
255        self._matchers = matchers
256
257    # The results
258    @property
259    def results(self):
260        return self._results
261
262    @results.setter
263    def results(self, results):
264        self._results = results
265
266    # The matchers
267    @property
268    def matchers(self):
269        return self._matchers
270
271    @matchers.setter
272    def matchers(self, matchers):
273        self._matchers = matchers
274
275    # The results
276    @property
277    def results(self):
278        return self._results
279
280    @results.setter
281    def results(self, results):
282        self._results = results
283
284    # The matchers
285    @property
286    def matchers(self):
287        return self._matchers
288
289    @matchers.setter
290    def matchers(self, matchers):
291        self._matchers = matchers
292
293    # The results
294    @property
295    def results(self):
296        return self._results
297
298    @results.setter
299    def results(self, results):
300        self._results = results
301
302    # The matchers
303    @property
304    def matchers(self):
305        return self._matchers
306
307    @matchers.setter
308    def matchers(self, matchers):
309        self._matchers = matchers
310
311    # The results
312    @property
313    def results(self):
314        return self._results
315
316    @results.setter
317    def results(self, results):
318        self._results = results
319
320    # The matchers
321    @property
322    def matchers(self):
323        return self._matchers
324
325    @matchers.setter
326    def matchers(self, matchers):
327        self._matchers = matchers
328
329    # The results
330    @property
331    def results(self):
332        return self._results
333
334    @results.setter
335    def results(self, results):
336        self._results = results
337
338    # The matchers
339    @property
340    def matchers(self):
341        return self._matchers
342
343    @matchers.setter
344    def matchers(self, matchers):
345        self._matchers = matchers
346
347    # The results
348    @property
349    def results(self):
350        return self._results
351
352    @results.setter
353    def results(self, results):
354        self._results = results
355
356    # The matchers
357    @property
358    def matchers(self):
359        return self._matchers
360
361    @matchers.setter
362    def matchers(self, matchers):
363        self._matchers = matchers
364
365    # The results
366    @property
367    def results(self):
368        return self._results
369
370    @results.setter
371    def results(self, results):
372        self._results = results
373
374    # The matchers
375    @property
376    def matchers(self):
377        return self._matchers
378
379    @matchers.setter
380    def matchers(self, matchers):
381        self._matchers = matchers
382
383    # The results
384    @property
385    def results(self):
386        return self._results
387
388    @results.setter
389    def results(self, results):
390        self._results = results
391
392    # The matchers
393    @property
394    def matchers(self):
395        return self._matchers
396
397    @matchers.setter
398    def matchers(self, matchers):
399        self._matchers = matchers
400
401    # The results
402    @property
403    def results(self):
404        return self._results
405
406    @results.setter
407    def results(self, results):
408        self._results = results
409
410    # The matchers
411    @property
412    def matchers(self):
413        return self._matchers
414
415    @matchers.setter
416    def matchers(self, matchers):
417        self._matchers = matchers
418
419    # The results
420    @property
421    def results(self):
422        return self._results
423
424    @results.setter
425    def results(self, results):
426        self._results = results
427
428    # The matchers
429    @property
430    def matchers(self):
431        return self._matchers
432
433    @matchers.setter
434    def matchers(self, matchers):
435        self._matchers = matchers
436
437    # The results
438    @property
439    def results(self):
440        return self._results
441
442    @results.setter
443    def results(self, results):
444        self._results = results
445
446    # The matchers
447    @property
448    def matchers(self):
449        return self._matchers
450
451    @matchers.setter
452    def matchers(self, matchers):
453        self._matchers = matchers
454
455    # The results
456    @property
457    def results(self):
458        return self._results
459
460    @results.setter
461    def results(self, results):
462        self._results = results
463
464    # The matchers
465    @property
466    def matchers(self):
467        return self._matchers
468
469    @matchers.setter
470    def matchers(self, matchers):
471        self._matchers = matchers
472
473    # The results
474    @property
475    def results(self):
476        return self._results
477
478    @results.setter
479    def results(self, results):
480        self._results = results
481
482    # The matchers
483    @property
484    def matchers(self):
485        return self._matchers
486
487    @matchers.setter
488    def matchers(self, matchers):
489        self._matchers = matchers
490
491    # The results
492    @property
493    def results(self):
494        return self._results
495
496    @results.setter
497    def results(self, results):
498        self._results = results
499
500    # The matchers
501    @property
502    def matchers(self):
503        return self._matchers
504
505    @matchers.setter
506    def matchers(self, matchers):
507        self._matchers = matchers
508
509    # The results
510    @property
511    def results(self):
512        return self._results
513
514    @results.setter
515    def results(self, results):
516        self._results = results
517
518    # The matchers
519    @property
520    def matchers(self):
521        return self._matchers
522
523    @matchers.setter
524    def matchers(self, matchers):
525        self._matchers = matchers
526
527    # The results
528    @property
529    def results(self):
530        return self._results
531
532    @results.setter
533    def results(self, results):
534        self._results = results
535
536    # The matchers
537    @property
538    def matchers(self):
539        return self._matchers
540
541    @matchers.setter
542    def matchers(self, matchers):
543        self._matchers = matchers
544
545    # The results
546    @property
547    def results(self):
548        return self._results
549
550    @results.setter
551    def results(self, results):
552        self._results = results
553
554    # The matchers
555    @property
556    def matchers(self):
557        return self._matchers
558
559    @matchers.setter
560    def matchers(self, matchers):
561        self._matchers = matchers
562
563    # The results
564    @property
565    def results(self):
566        return self._results
567
568    @results.setter
569    def results(self, results):
570        self._results = results
571
572    # The matchers
573    @property
574    def matchers(self):
575        return self._matchers
576
577    @matchers.setter
578    def matchers(self, matchers):
579        self._matchers = matchers
580
581    # The results
582    @property
583    def results(self):
584        return self._results
585
586    @results.setter
587    def results(self, results):
588        self._results = results
589
590    # The matchers
591    @property
592    def matchers(self):
593        return self._matchers
594
595    @matchers.setter
596    def matchers(self, matchers):
597        self._matchers = matchers
598
599    # The results
600    @property
601    def results(self):
602        return self._results
603
604    @results.setter
605    def results(self, results):
606        self._results = results
607
608    # The matchers
609    @property
610    def matchers(self):
611        return self._matchers
612
613    @matchers.setter
614    def matchers(self, matchers):
615        self._matchers = matchers
616
617    # The results
618    @property
619    def results(self):
620        return self._results
621
622    @results.setter
623    def results(self, results):
624        self._results = results
625
626    # The matchers
627    @property
628    def matchers(self):
629        return self._matchers
630
631    @matchers.setter
632    def matchers(self, matchers):
633        self._matchers = matchers
634
635    # The results
636    @property
637    def results(self):
638        return self._results
639
640    @results.setter
641    def results(self, results):
642        self._results = results
643
644    # The matchers
645    @property
646    def matchers(self):
647        return self._matchers
648
649    @matchers.setter
650    def matchers(self, matchers):
651        self._matchers = matchers
652
653    # The results
654    @property
655    def results(self):
656        return self._results
657
658    @results.setter
659    def results(self, results):
660        self._results = results
661
662    # The matchers
663    @property
664    def matchers(self):
665        return self._matchers
666
667    @matchers.setter
668    def matchers(self, matchers):
669        self._matchers = matchers
670
671    # The results
672    @property
673    def results(self):
674        return self._results
675
676    @results.setter
677    def results(self, results):
678        self._results = results
679
680    # The matchers
681    @property
682    def matchers(self):
683        return self._matchers
684
685    @matchers.setter
686    def matchers(self, matchers):
687        self._matchers = matchers
688
689    # The results
690    @property
691    def results(self):
692        return self._results
693
694    @results.setter
695    def results(self, results):
696        self._results = results
697
698    # The matchers
699    @property
700    def matchers(self):
701        return self._matchers
702
703    @matchers.setter
704    def matchers(self, matchers):
705        self._matchers = matchers
706
707    # The results
708    @property
709    def results(self):
710        return self._results
711
712    @results.setter
713    def results(self, results):
714        self._results = results
715
716    # The matchers
717    @property
718    def matchers(self):
719        return self._matchers
720
721    @matchers.setter
722    def matchers(self, matchers):
723        self._matchers = matchers
724
725    # The results
726    @property
727    def results(self):
728        return self._results
729
730    @results.setter
731    def results(self, results):
732        self._results = results
733
734    # The matchers
735    @property
736    def matchers(self):
737        return self._matchers
738
739    @matchers.setter
740    def matchers(self, matchers):
741        self._matchers = matchers
742
743    # The results
744    @property
745    def results(self):
746        return self._results
747
748    @results.setter
749    def results(self, results):
750        self._results = results
751
752    # The matchers
753    @property
754    def matchers(self):
755        return self._matchers
756
757    @matchers.setter
758    def matchers(self, matchers):
759        self._matchers = matchers
760
761    # The results
762    @property
763    def results(self):
764        return self._results
765
766    @results.setter
767    def results(self, results):
768        self._results = results
769
770    # The matchers
771    @property
772    def matchers(self):
773        return self._matchers
774
775    @matchers.setter
776    def matchers(self, matchers):
777        self._matchers = matchers
778
779    # The results
780    @property
781    def results(self):
782        return self._results
783
784    @results.setter
785    def results(self, results):
786        self._results = results
787
788    # The matchers
789    @property
790    def matchers(self):
791        return self._matchers
792
793    @matchers.setter
794    def matchers(self, matchers):
795        self._matchers = matchers
796
797    # The results
798    @property
799    def results(self):
800        return self._results
801
802    @results.setter
803    def results(self, results):
804        self._results = results
805
806    # The matchers
807    @property
808    def matchers(self):
809        return self._matchers
810
811    @matchers.setter
812    def matchers(self, matchers):
813        self._matchers = matchers
814
815    # The results
816    @property
817    def results(self):
818        return self._results
819
820    @results.setter
821    def results(self, results):
822        self._results = results
823
824    # The matchers
825    @property
826    def matchers(self):
827        return self._matchers
828
829    @matchers.setter
830    def matchers(self, matchers):
831        self._matchers = matchers
832
833    # The results
834    @property
835    def results(self):
836        return self._results
837
838    @results.setter
839    def results(self, results):
840        self._results = results
841
842    # The matchers
843    @property
844    def matchers(self):
845        return self._matchers
846
847    @matchers.setter
848    def matchers(self, matchers):
849        self._matchers = matchers
850
851    # The results
852    @property
853    def results(self):
854        return self._results
855
856    @results.setter
857    def results(self, results):
858        self._results = results
859
860    # The matchers
861    @property
862    def matchers(self):
863        return self._matchers
864
865    @matchers.setter
866    def matchers(self, matchers):
867        self._matchers = matchers
868
869    # The results
870    @property
871    def results(self):
872        return self._results
873
874    @results.setter
875    def results(self, results):
876        self._results = results
877
878    # The matchers
879    @property
880    def matchers(self):
881        return self._matchers
882
883    @matchers.setter
884    def matchers(self, matchers):
885        self._matchers = matchers
886
887    # The results
888    @property
889    def results(self):
890        return self._results
891
892    @results.setter
893    def results(self, results):
894        self._results = results
895
896    # The matchers
897    @property
898    def matchers(self):
899        return self._matchers
900
901    @matchers.setter
902    def matchers(self, matchers):
903        self._matchers = matchers
904
905    # The results
906    @property
907    def results(self):
908        return self._results
909
910    @results.setter
911    def results(self, results):
912        self._results = results
913
914    # The matchers
915    @property
916    def matchers(self):
917        return self._matchers
918
919    @matchers.setter
920    def matchers(self, matchers):
921        self._matchers = matchers
922
923    # The results
924    @property
925    def results(self):
926        return self._results
927
928    @results.setter
929    def results(self, results):
930        self._results = results
931
932    # The matchers
933    @property
934    def matchers(self):
935        return self._matchers
936
937    @matchers.setter
938    def matchers(self, matchers):
939        self._matchers = matchers
940
941    # The results
942    @property
943    def results(self):
944        return self._results
945
946    @results.setter
947    def results(self, results):
948        self._results = results
949
950    # The matchers
951    @property
952    def matchers(self):
953        return self._matchers
954
955    @matchers.setter
956    def matchers(self, matchers):
957        self._matchers = matchers
958
959    # The results
960    @property
961    def results(self):
962        return self._results
963
964    @results.setter
965    def results(self, results):
966        self._results = results
967
968    # The matchers
969    @property
970    def matchers(self):
971        return self._matchers
972
973    @matchers.setter
974    def matchers(self, matchers):
975        self._matchers = matchers
976
977    # The results
978    @property
979    def results(self):
980        return self._results
981
982    @results.setter
983    def results(self, results):
984        self._results = results
985
986    # The matchers
987    @property
988    def matchers(self):
989        return self._matchers
990
991    @matchers.setter
992    def matchers(self, matchers):
993        self._matchers = matchers
994
995    # The results
996    @property
997    def results(self):
998        return self._results
999
1000   @results.setter
1001   def results(self, results):
1002       self._results = results
1003
1004   # The matchers
1005   @property
1006   def matchers(self):
1007       return self._matchers
1008
1009   @matchers.setter
1010   def matchers(self, matchers):
1011       self._matchers = matchers
1012
1013   # The results
1014   @property
1015   def results(self):
1016       return self._results
1017
1018   @results.setter
1019   def results(self, results):
1020       self._results = results
1021
1022   # The matchers
1023   @property
1024   def matchers(self):
1025       return self._matchers
1026
1027   @matchers.setter
1028   def matchers(self, matchers):
1029       self._matchers = matchers
1030
1031   # The results
1032   @property
1033   def results(self):
1034       return self._results
1035
1036   @results.setter
1037   def results(self, results):
1038       self._results = results
1039
1040   # The matchers
1041   @property
1042   def matchers(self):
1043       return self._matchers
1044
1045   @matchers.setter
1046   def matchers(self, matchers):
1047       self._matchers = matchers
1048
1049   # The results
1050   @property
1051   def results(self):
1052       return self._results
1053
1054   @results.setter
1055   def results(self, results):
1056       self._results = results
1057
1058   # The matchers
1059   @property
1060   def matchers(self):
1061       return self._matchers
1062
1063   @matchers.setter
1064   def matchers(self, matchers):
1065       self._matchers = matchers
1066
1067   # The results
1068   @property
1069   def results(self):
1070       return self._results
1071
1072   @results.setter
1073   def results(self, results):
1074       self._results = results
1075
1076   # The matchers
1077   @property
1078   def matchers(self):
1079       return self._matchers
1080
1081   @matchers.setter
1082   def matchers(self, matchers):
1083       self._matchers = matchers
1084
1085   # The results
1086   @property
1087   def results(self):
1088       return self._results
1089
1090   @results.setter
1091   def results(self, results):
1092       self._results = results
1093
1094   # The matchers
1095   @property
1096   def matchers(self):
1097       return self._matchers
1098
1099   @matchers.setter
1100   def matchers(self, matchers):
1101       self._matchers = matchers
1102
1103   # The results
1104   @property
1105   def results(self):
1106       return self._results
1107
1108   @results.setter
1109   def results(self, results):
1110       self._results = results
1111
1112   # The matchers
1113   @property
1114   def matchers(self):
1115       return self._matchers
1116
1117   @matchers.setter
1118   def matchers(self, matchers):
1119       self._matchers = matchers
1120
1121   # The results
1122   @property
1123   def results(self):
1124       return self._results
1125
1126   @results.setter
1127   def results(self, results):
1128       self._results = results
1129
1130   # The matchers
1131   @property
1132   def matchers(self):
1133       return self._matchers
1134
1135   @matchers.setter
1136   def matchers(self, matchers):
1137       self._matchers = matchers
1138
1139   # The results
1140   @property
1141   def results(self):
1142       return self._results
1143
1144   @results.setter
1145   def results(self, results):
1146       self._results = results
1147
1148   # The matchers
1149   @property
1150   def matchers(self):
1151       return self._matchers
1152
1153   @matchers.setter
1154   def matchers(self, matchers):
1155       self._matchers = matchers
1156
1157   # The results
1158   @property
1159   def results(self):
1160       return self._results
1161
1162   @results.setter
1163   def results(self, results):
1164       self._results = results
1165
1166   # The matchers
1167   @property
1168   def matchers(self):
1169       return self._matchers
1170
1171   @matchers.setter
1172   def matchers(self, matchers):
1173       self._matchers = matchers
1174
1175   # The results
1176   @property
1177   def results(self):
1178       return self._results
1179
1180   @results.setter
1181   def results(self, results):
1182       self._results = results
1183
1184   # The matchers
1185   @property
1186   def matchers(self):
1187       return self._matchers
1188
1189   @matchers.setter
1190   def matchers(self, matchers):
1191       self._matchers = matchers
1192
1193   # The results
1194   @property
1195   def results(self):
1196       return self._results
1197
1198   @results.setter
1199   def results(self, results):
1200       self._results = results
1201
1202   # The matchers
1203   @property
1204   def matchers(self):
1205       return self._matchers
1206
1207   @matchers.setter
1208   def matchers(self, matchers):
1209       self._matchers = matchers
1210
1211   # The results
1212   @property
1213   def results(self):
1214       return self._results
1215
1216   @results.setter
1217   def results(self, results):
1218       self._results = results
1219
1220   # The matchers
1221   @property
1222   def matchers(self):
1223       return self._matchers
1224
1225   @matchers.setter
1226   def matchers(self, matchers):
1227       self._matchers = matchers
1228
1229   # The results
1230   @property
1231   def results(self):
1232       return self._results
1233
1234   @results.setter
1235   def results(self, results):
1236       self._results = results
1237
1238   # The matchers
1239   @property
1240   def matchers(self):
1241       return self._matchers
1242
1243   @matchers.setter
1244   def matchers(self, matchers):
1245       self._matchers = matchers
1246
1247   # The results
1248   @property
1249   def results(self):
1250       return self._results
1251
1252   @results.setter
1253   def results(self, results):
1254       self._results = results
1255
1256   # The matchers
1257   @property
1258   def matchers(self):
1259       return self._matchers
1260
1261   @matchers.setter
1262   def matchers(self, matchers):
1263       self._matchers = matchers
1264
1265   # The results
1266   @property
1267   def results(self):
1268       return self._results
1269
1270   @results.setter
1271   def results(self, results):
1272       self._results = results
1273
1274   # The matchers
1275   @property
1276   def matchers(self):
1277       return self._matchers
1278
1279   @matchers.setter
1280   def matchers(self, matchers):
1281       self._matchers = matchers
1282
1283   # The results
1284   @property
1285   def results(self):
1286       return self._results
1287
1288   @results.setter
1289   def results(self, results):
1290       self._results = results
1291
1292   # The matchers
1293   @property
1294   def matchers(self):
1295       return self._matchers
1296
1297   @matchers.setter
1298   def matchers(self, matchers):
1299       self._matchers = matchers
1300
1301   # The results
1302   @property
1303   def results(self):
1304       return self._results
1305
1306   @results.setter
1307   def results(self, results):
1308       self._results = results
1309
1310   # The matchers
1311   @property
1312   def matchers(self):
1313       return self._matchers
1314
1315   @matchers.setter
1316   def matchers(self, matchers):
1317       self._matchers = matchers
1318
1319   # The results
1320   @property
1321   def results(self):
1322       return self._results
1323
1324   @results.setter
1325   def results(self, results):
1326       self._results = results
1327
1328   # The matchers
1329   @property
1330   def matchers(self):
1331       return self._matchers
1332
1333   @matchers.setter
1334   def matchers(self, matchers):
1335       self._matchers = matchers
1336
1337   # The results
1338   @property
1339   def results(self):
1340       return self._results
1341
1342   @results.setter
1343   def results(self, results):
1344       self._results = results
1345
1346   # The matchers
1347   @property
1348   def matchers(self):
1349       return self._matchers
1350
1351   @matchers.setter
1352   def matchers(self, matchers):
1353       self._matchers = matchers
1354
1355   # The results
1356   @property
1357   def results(self):
1358       return self._results
1359
1360   @results.setter
1361   def results(self, results):
1362       self._results = results
1363
1364   # The matchers
1365   @property
1366   def matchers(self):
1367       return self._matchers
1368
1369   @matchers.setter
1370   def matchers(self, matchers):
1371       self._matchers = matchers
1372
1373   # The results
1374   @property
1375   def results(self):
1376       return self._results
1377
1378   @results.setter
1379   def results(self, results):
1380       self._results = results
1381
1382   # The matchers
1383   @property
1384   def matchers(self):
1385       return self._matchers
1386
1387   @matchers.setter
1388   def matchers(self, matchers):
1389       self._matchers = matchers
1390
1391   # The results
1392   @property
1393   def results(self):
1394       return self._results
1395
1396   @results.setter
1397   def results(self, results):
1398       self._results = results
1399
1400   # The matchers
1401   @property
1402   def matchers(self):
1403       return self._matchers
1404
1405   @matchers.setter
1406   def matchers(self, matchers):
1407       self._matchers = matchers
1408
1409   # The results
1410   @property
1411   def results(self):
1412       return self._results
1413
1414   @results.setter
1415   def results(self, results):
1416       self._results = results
1417
1418   # The matchers
1419   @property
1420   def matchers(self):
1421       return self._matchers
1422
1423   @matchers.setter
1424   def matchers(self, matchers):
1425       self._matchers = matchers
1426
1427   # The results
1428   @property
1429   def results(self):
1430       return self._results
1431
1432   @results.setter
1433   def results(self, results):
1434       self._results = results
1435
1436   # The matchers
1437   @property
1438   def matchers(self):
1439       return self._matchers
1440
1441   @matchers.setter
1442   def matchers(self, matchers):
1443       self._matchers = matchers
1444
1445   # The results
1446   @property
1447   def results(self):
1448       return self._results
1449
1450   @results.setter
1451   def results(self, results):
1
```

3.4.2 Memory Analysis - Volatility

If any of the hunting activities identified a threat, Volatility provides a wide range of modules that will help you extract or rather, carve out of the memory dump all of the malicious object(s) (process, driver, ...) for further analysis.



3.4.2 Memory Analysis - Volatility

Further details on how to use Volatility are available on its Wiki page, [here](#).

Lastly, if you want to play with some memory samples and perfect your Volatility knowledge, you can download them from [here](#).

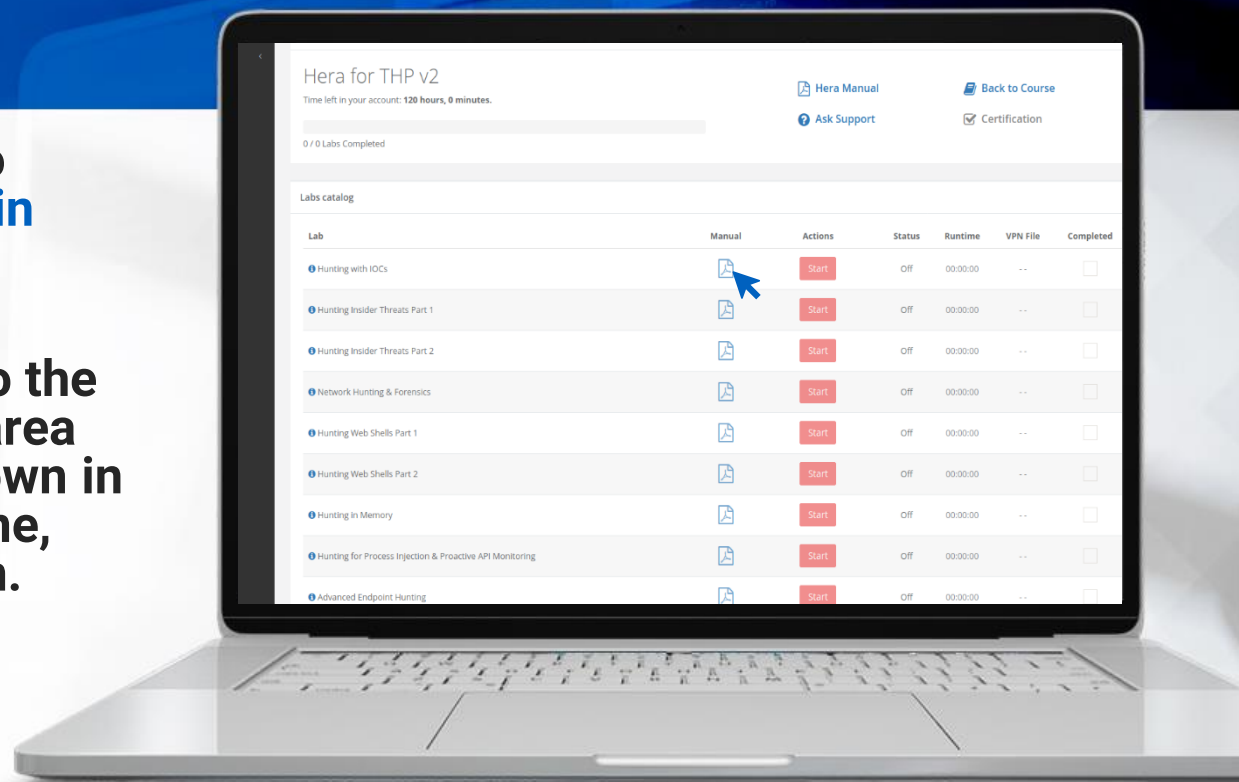
```
20 with open('result.txt', 'w') as f:
21     f.write('Result: %s\n' % result)
22
23 # Main: Create the experiment
24 # -----
25 # @param experiment_name: the experiment name (e.g. 'Control')
26 # @param observations: an array of Observations, or None
27 # @param control: the control Observation
28
29 def initialize(experiment, observations = None, control = None):
30     @experiment = experiment
31     @observations = observations
32     @control = control
33     @candidates = observations - [control]
34     evaluate_candidates()
35
36 # Create the experiment object
37 # -----
38 # @param experiment_name: the name of the experiment
39
40 def experiment_name(experiment_name):
41     experiment_name = experiment_name
42
43 # Check the result a match between all samples
44 def match_result(result):
45     result = result
46
47 # Check the result a match between all samples
48 def match_result(result):
49     result = result
50
51 # Check the result a match between all samples
52 def match_result(result):
53     result = result
```

3.4.2.1 Hera Lab

Put what you've learned to practice with the **Hunting in Memory** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click **LINK**.



***NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

3.4.3 Live System Memory Hunting

Unfortunately, getting a memory dump (from all your systems) and performing analysis on it is rather impractical. It is too time consuming, and therefore the hunts are performed on a subset of hosts only.

Another obstacle is the memory size – on average, the size is 16GB from workstations and commonly 64 GB (or more) on servers.



3.4.3 Live System Memory Hunting

Recently, some tools have emerged that attempt to scale memory hunting, primarily focusing on detecting injected code on the live machine without the need of obtaining memory dumps. The tools we'll look into are:

- [GetInjectedThread.ps1](#)
- [Memhunter](#)
- [Captain](#)

<https://gist.github.com/jaredcatkinson/23905d34537ce4b5b1818c3e6405c1d2>

<https://github.com/marcosd4h/memhunter>

<https://github.com/y3n11/Captain>

```
24 # the experiment's result is a list of candidates
25 # the experiment's result is a list of candidates
26 # the experiment's result is a list of candidates
27
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - @control
33   evaluate_candidates
34
35   freeze
36 end
37
38 # Returns the experiment's context
39 def context
40   @experiment.context
41 end
42
43 # Returns the name of the experiment
44 def experiment_name
45   @experiment.name
46 end
47
48 # Returns the result a match between all candidates
49 def match
50   @candidates.result
51 end
```

3.4.3.1 Live System Memory Hunting - Get-InjectedThread

Get-InjectedThread is defined by the author as a tool that can detect:

- Classic Injection
- Reflective DLL Injection
- Memory Module (similar technique to RDI)

The original presentation of the tool is available [here](#).

```
21 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
22 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
23 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
24 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
25 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
26 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
27 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
28 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
29 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
30 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
31 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
32 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
33 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
34 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
35 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
36 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
37 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
38 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
39 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
40 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
41 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
42 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
43 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
44 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
45 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
46 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
47 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
48 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
49 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
50 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
51 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
52 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
53 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
54 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
55 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
56 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
57 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
58 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
59 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
60 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
61 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
62 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
63 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
64 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
65 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
66 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
67 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
68 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
69 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
70 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
71 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
72 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
73 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
74 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
75 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
76 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
77 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
78 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
79 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
80 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
81 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
82 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
83 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
84 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
85 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
86 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
87 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
88 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
89 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
90 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
91 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
92 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
93 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
94 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
95 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
96 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
97 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
98 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
99 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
100 # [DllImport] [MarshalAs(UnmanagedType.LPStr)] [Unmanaged]
```

3.4.3.1 Live System Memory Hunting - Get-InjectedThread

Running the script on a compromised host returns confirmation of the injection and additional information about the process and thread detected.

```
PS C:\Tools> Get-InjectedThread

ProcessName      : rundll32.exe
ProcessId        : 3624
Path             : C:\Windows\system32\rundll32.exe
KernelPath       : C:\Windows\System32\rundll32.exe
CommandLine      : rundll32.exe
PathMismatch     : False
ThreadId        : 1288
AllocatedMemoryProtection : PAGE_EXECUTE_READWRITE
MemoryProtection : PAGE_EXECUTE_READWRITE
MemoryState      : MEM_COMMIT
MemoryType       : MEM_PRIVATE
BasePriority      : 4
IsUniqueThreadToken : False
Integrity        : MEDIUM_MANDATORY_LEVEL
Privilege        : SeChangeNotifyPrivilege
LogonId          : 999
SecurityIdentifier : S-1-5-21-1196709796-4176271787-2474106595-1000
Username         : WIN-HIHVB9J997R\SYSTEM
LogonSessionStartTime : 9/25/2017 6:32:47 PM
LogonType        : System
AuthenticationPackage : NTLM
BaseAddress      : 8656005
Size             : 143360
Bytes            : {85, 139, 236, 131...}
```


3.4.3.2 Live System Memory Hunting - Memhunter

```
C:\tools\memhunter>memhunter.exe -r -m 1

===== New Suspicious Process Found: C:\Windows\System32\notepad.exe =====

Hunter Module: SuspiciousThreads
Process ID: 11496
Process Name: notepad.exe
Process Path: c:\windows\system32\notepad.exe
Process Command Line: notepad
Process Nr Threads: 8
Process Base Priority: 32
Suspicious Module Name: SuspiciousThreads
Suspicious Module Path: memory
Suspicious Thread ID: 7688
Suspicious Thread Priority: 0
Suspicious Thread Base Addr: 0x0000015CA8170410
Suspicious Memory Protection: PAGE_EXECUTE_READWRITE
Suspicious Memory Alloc Protection: PAGE_EXECUTE_READWRITE
Suspicious Memory State: MEM_COMMIT
Suspicious Memory Type: MEM_PRIVATE
```

A working PoC video of the tool is available [here](https://www.youtube.com/watch?v=t_fR1sCENkc).

3.4.3.3 Live System Memory Hunting - Captain

Captain is an endpoint monitoring tool that is designed to spot malicious events through API hooking.

Captain, among others, is cable of detecting:

- Code Injection
- Memory dump creation (e.g. dump of LSASS)
- Fileless malware
- Execution of Office macros

```
21 def __init__(self, experiment_name, control_name):
22     self.experiment_name = experiment_name
23     self.control_name = control_name
24     self.experiment = Experiment(experiment_name)
25     self.control = Control(control_name)
26     self.observations = []
27     self.candidates = []
28     self.evaluate_candidates()
29
30 def evaluate_candidates(self):
31     # Evaluate the candidates
32     for candidate in self.candidates:
33         # Evaluate the candidate
34         candidate.evaluate()
35         # Add the candidate to the observations
36         self.observations.append(candidate)
37
38 def freeze(self):
39     # Freeze the experiment
40     self.experiment.freeze()
41
42 def __str__(self):
43     return f"Experiment: {self.experiment_name}, Control: {self.control_name}"
44
45 def __repr__(self):
46     return f"Experiment: {self.experiment_name}, Control: {self.control_name}"
47
48 def __eq__(self, other):
49     return self.experiment_name == other.experiment_name and self.control_name == other.control_name
50
51 def __ne__(self, other):
52     return not self.__eq__(other)
53
54 def __lt__(self, other):
55     return self.experiment_name < other.experiment_name
56
57 def __gt__(self, other):
58     return self.experiment_name > other.experiment_name
59
60 def __le__(self, other):
61     return self.experiment_name <= other.experiment_name
62
63 def __ge__(self, other):
64     return self.experiment_name >= other.experiment_name
65
66 def __hash__(self):
67     return hash(self.experiment_name + self.control_name)
```



3.4.3.3 Live System Memory Hunting - Captain

Captain requires its 4 components to operate:

- **Monitor.ps1** – Monitors for process creations and injects Captain.dll in new processes
- **Injector.exe** – Used for the injection of Captain.dll
- **Captain.dll** – Hooks Windows API functions and outputs events
- **Behan.py** – analyzes the events captured by Captain.dll (based on provided signatures for alerting)

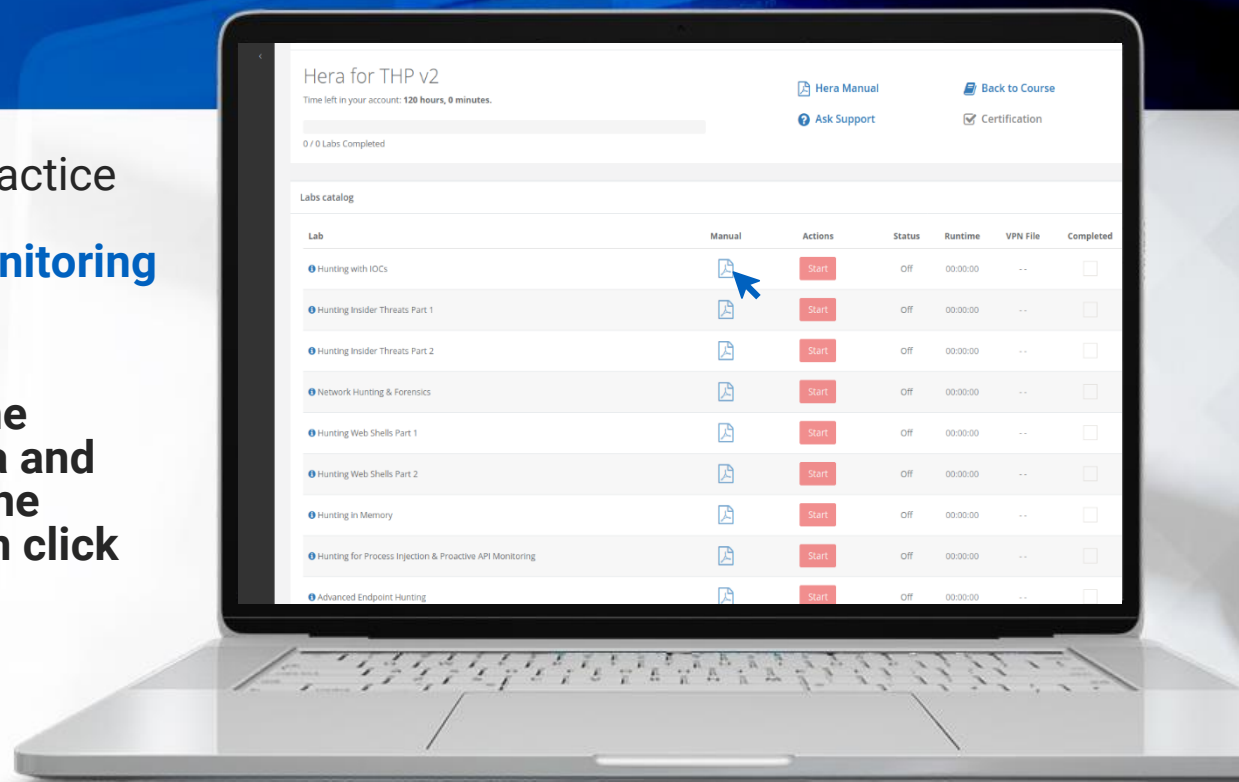


3.4.4 Hera Lab

Put what you've learned to practice with the **Hunting for Process Injection & Proactive API Monitoring** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).



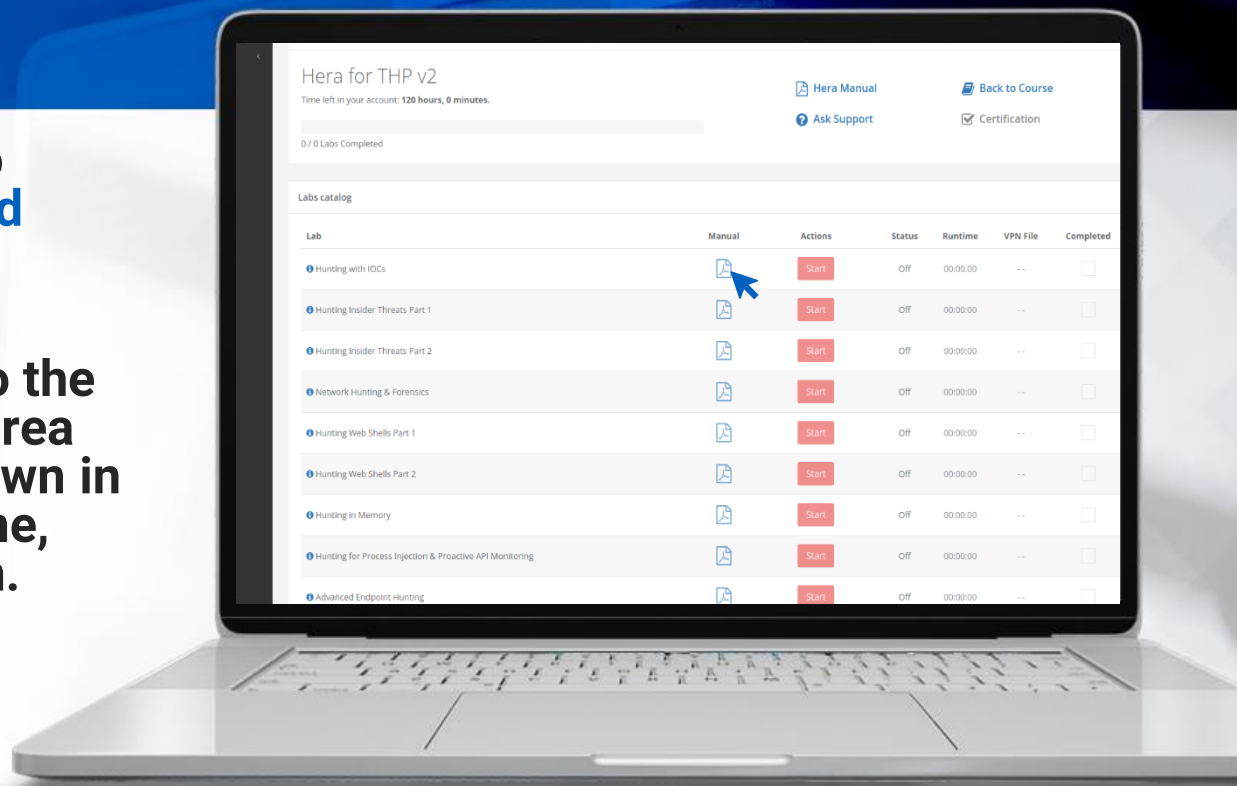
***NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

3.4.5 Hera Lab

Put what you've learned to practice with the **Advanced Endpoint Hunting** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click **[LINK](#)**.



***NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

Malware Analysis



3.5 Malware Analysis

Even though malware analysis is beyond the scope of this course, it's still worth a mention.

Malware analysis is needed when a binary needs to be analyzed further. We know that malware, whether it's packed, encrypted, etc., is in clear-text in memory, but in order to further understand the malware, analysis is needed.



3.5 Malware Analysis

If your security team doesn't have a dedicated malware analyst, then as a threat hunter, this is a skill to have. Even if it's basic malware analysis skills, it will be helpful.

A threat hunter is similar to a spec ops operator. No matter what he/she encounters, he/she is trained and has the skill to complete the task. Whether it is inspecting network traffic, hunting for malicious files in various operating systems, performing incident response, memory analysis, etc., they are ready.

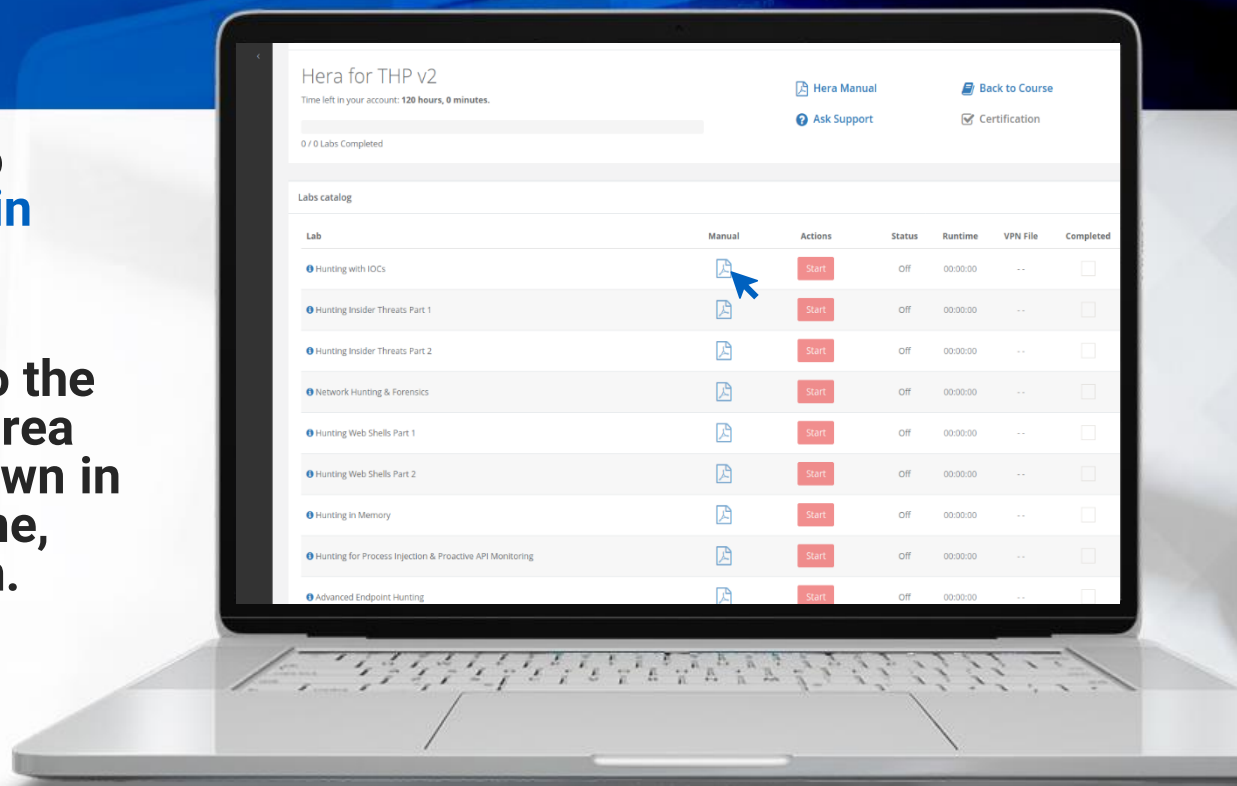


3.5.1 Hera Lab

Put what you've learned to practice with the **Hunting in Malware Part 1** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click **[LINK](#)**.



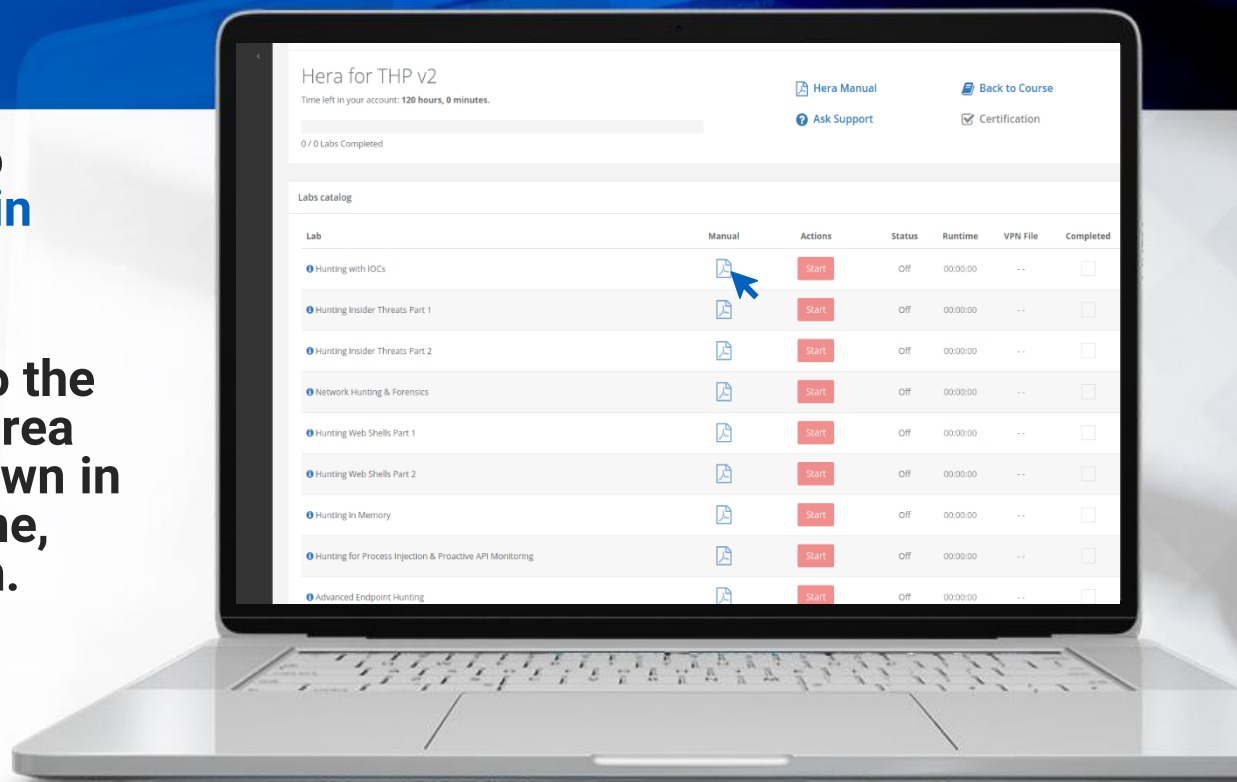
***NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

3.5.2 Hera Lab

Put what you've learned to practice with the **Hunting in Malware Part 2** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click **[LINK](#)**.



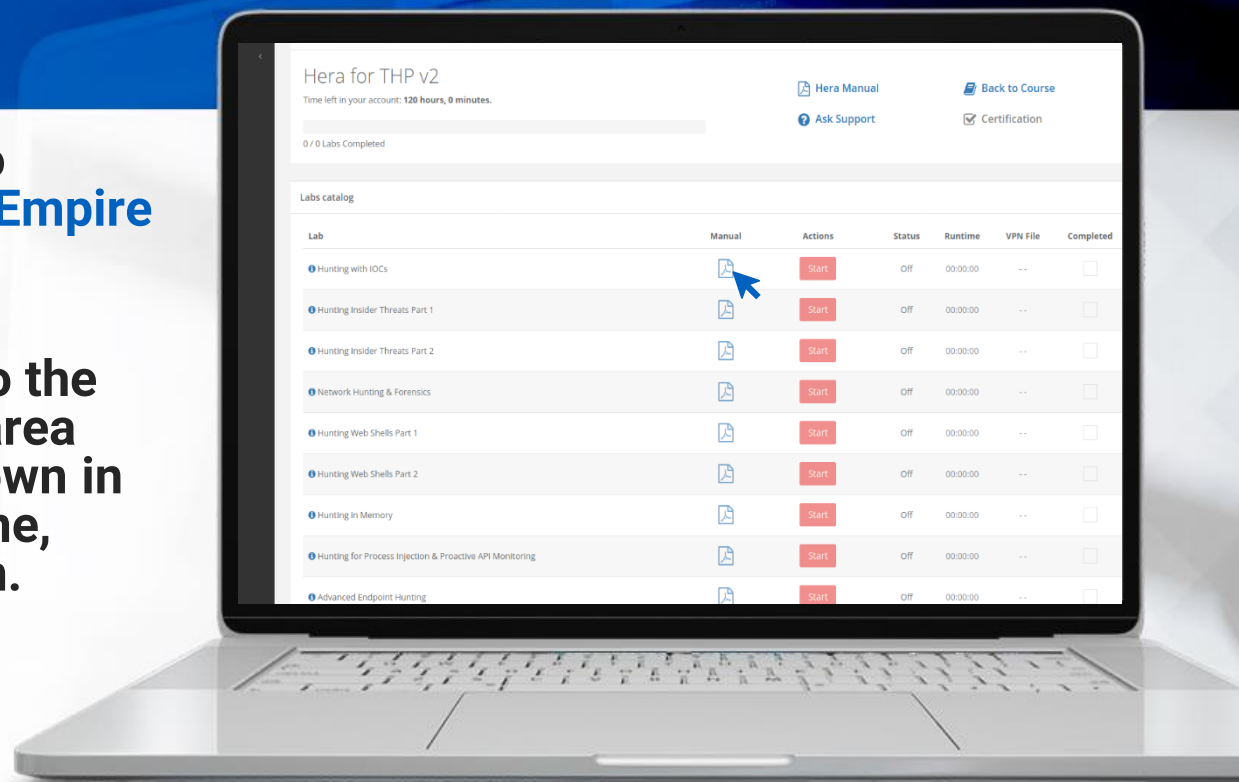
***NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

3.5.3 Hera Lab

Put what you've learned to practice with the **Hunting Empire** lab!

To **ACCESS** your lab, go to the course in your members area and click the labs drop-down in the appropriate module line, then click the manual icon.

All labs are only available in Full or Elite Editions of the course. To upgrade, click **[LINK](#)**.



***NOTE:** some courses contain several labs and manuals, please make sure to click the file icon as it may be a zip that contains multiple lab manuals.

Conclusion

This concludes this module on Hunting Malware.

We have covered:

- ✓ Various detection tools
- ✓ Various detection techniques
- ✓ Memory analysis tools
- ✓ The importance of malware analysis



References



References

[PE Capture v1.2](http://www.novirusthanks.org/products/pe-capture/)

<http://www.novirusthanks.org/products/pe-capture/>

[PE Capture Service v1.2](http://www.novirusthanks.org/products/pe-capture-service/)

<http://www.novirusthanks.org/products/pe-capture-service/>

[RandomCode](https://github.com/abhisek/RandomCode/tree/master/Malware/Process)

<https://github.com/abhisek/RandomCode/tree/master/Malware/Process>

[Meterpreter_Payload_Detection](https://github.com/DamonMohammadbagher/Meterpreter_Payload_Detection)

https://github.com/DamonMohammadbagher/Meterpreter_Payload_Detection



References

[reflective-injection-detection](https://github.com/papadp/reflective-injection-detection)

<https://github.com/papadp/reflective-injection-detection>

[PowerShellArsenal](https://github.com/mattifestation/PowerShellArsenal)

<https://github.com/mattifestation/PowerShellArsenal>

[NtQueryInformationThread function](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684283(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684283\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684283(v=vs.85).aspx)

[Get-InjectedThread.ps1](https://gist.github.com/jaredcatkinson/23905d34537ce4b5b1818c3e6405c1d2)

<https://gist.github.com/jaredcatkinson/23905d34537ce4b5b1818c3e6405c1d2>



References

[ssdeep - Fuzzy hashing program](https://ssdeep-project.github.io/ssdeep/index.html)

<https://ssdeep-project.github.io/ssdeep/index.html>

[Identifying Almost Identical Files Using Context Triggered Pieewise Hashing](http://dfrws.org/sites/default/files/session-files/paper-identifying_almost_identical_files_using_context_triggered_pieewise_hashing.pdf)

http://dfrws.org/sites/default/files/session-files/paper-identifying_almost_identical_files_using_context_triggered_pieewise_hashing.pdf

[\[How To\] Fuzzy Hashing with SSDEEP \(similarity matching\)](https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-(similarity-matching).html)

[https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-\(similarity-matching\).html](https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-(similarity-matching).html)

[ssdeep](https://github.com/ssdeep-project/ssdeep)

<https://github.com/ssdeep-project/ssdeep>



References

[VirusTotal += imphash](http://blog.virustotal.com/2014/02/virustotal-imphash.html)

<http://blog.virustotal.com/2014/02/virustotal-imphash.html>

[Tracking Malware with Import Hashing](https://www.fireeye.com/blog/threat-research/2014/01/tracking-malware-import-hashing.html)

<https://www.fireeye.com/blog/threat-research/2014/01/tracking-malware-import-hashing.html>

[ImpHash-Generator](https://github.com/Neo23x0/ImpHash-Generator)

<https://github.com/Neo23x0/ImpHash-Generator>

[Caching Out: The Value of Shimcache for Investigators](https://www.fireeye.com/blog/threat-research/2015/06/caching_out_the_val.html)

https://www.fireeye.com/blog/threat-research/2015/06/caching_out_the_val.html



References

[ShimCacheParser](#)

<https://github.com/mandiant/ShimCacheParser>

[Evolving Analytics for Execution Trace Data](#)

<https://www.fireeye.com/blog/threat-research/2017/04/appcompatprocessor.html>

[appcompatprocessor](#)

<https://github.com/mbevilacqua/appcompatprocessor>

[Reflective DLL Injection Detection through Memhunter](#)

https://www.youtube.com/watch?v=t_fR1sCENkc



References

[Hunting In Memory](https://www.sans.org/cyber-security-summit/archives/file/summit-archive-1492714038.pdf)

<https://www.sans.org/cyber-security-summit/archives/file/summit-archive-1492714038.pdf>

[FTK Imager](https://accessdata.com/product-download/ftk-imager-version-4-2-0)

<https://accessdata.com/product-download/ftk-imager-version-4-2-0>

[Comae Stardust](https://my.comae.com/)

<https://my.comae.com/>

[MAGNET RAM Capture](https://www.magnetforensics.com/resources/magnet-ram-capture/)

<https://www.magnetforensics.com/resources/magnet-ram-capture/>



References

[Redline](https://www.fireeye.com/services/freeware/redline.html)

<https://www.fireeye.com/services/freeware/redline.html>

[volatility](https://github.com/volatilityfoundation/volatility)

<https://github.com/volatilityfoundation/volatility>

[Captain](https://github.com/y3n11/Captain)

<https://github.com/y3n11/Captain>

[memhunter](https://github.com/marcosd4h/memhunter)

<https://github.com/marcosd4h/memhunter>



References

[Volatility – Memory Samples](#)

<https://github.com/volatilityfoundation/volatility/wiki/Memory-Samples>

[Volatility Usage](#)

<https://github.com/volatilityfoundation/volatility/wiki/Volatility-Usage>

[Malware Researcher's Handbook \(Demystifying PE File\)](#)

<https://resources.infosecinstitute.com/2-malware-researchers-handbook-demystifying-pe-file/#gref>

[DETECTING DECEPTIVE PROCESS HOLLOWING TECHNIQUES USING HOLLOWFIND VOLATILITY PLUGIN](#)

<https://cysinfo.com/detecting-deceptive-hollowing-techniques/>





References

[Ten process injection techniques: A technical survey of common and trending process injection techniques](https://www.endgame.com/blog/technical-blog/ten-process-injection-techniques-technical-survey-common-and-trending-process)

<https://www.endgame.com/blog/technical-blog/ten-process-injection-techniques-technical-survey-common-and-trending-process>

[yarascan](https://github.com/volatilityfoundation/volatility/wiki/Command-Reference-Mal#yarascan)

<https://github.com/volatilityfoundation/volatility/wiki/Command-Reference-Mal#yarascan>

[Finding Advanced Malware Using Volatility](https://eforensicsmag.com/finding-advanced-malware-using-volatility/)

<https://eforensicsmag.com/finding-advanced-malware-using-volatility/>



Videos

Here's a list of all videos in this module. To [ACCESS](#) your video, go to the course in your members area and click the resources drop-down in the appropriate module line.

Note that all videos are only available in Full or Elite Editions of the course. To upgrade, click [LINK](#).

Redline – Created Standard Collector

Redline – Basic Usage

Redline – Create Analysis File

Redline – Detecting Code Injection



Labs

Hunting in Memory

Lab 7.1: The organization you work for is asking you to perform memory threat hunting on a randomly selected machine. As a hunting exercise to keep you sharp, the IT Security manager tasked you specifically with looking for anomalous connections and memory injections.

Lab 7.2: The organization you work for is also asking you to perform memory threat hunting on a Linux machine. As a hunting exercise to keep you sharp, the IT Security manager tasked you specifically with looking for the existence of Linux rootkits.

Hunting for Process Injection & Proactive API Monitoring

Attackers love hiding/injecting malicious code into processes. In this lab, you will learn how to hunt for various process injection techniques and how to leverage userland API monitoring for more effective hunts.

****Labs are only available in Full or Elite Editions of the course. To [ACCESS](#) your labs, go to the course in your members area and click the labs drop-down in the appropriate module line. To UPGRADE to gain access, click [LINK](#).***



Labs

Advanced Endpoint Hunting

Inside THP you will find two (2) distinct labs on advanced hacking techniques hunting at the endpoint level. Specifically, you will learn how to hunt for process doppelganging, AMSI bypasses, parent PID spoofing, reflective DLL injection, module stomping etc.

Hunting Malware Part 1

Your manager, Tony, wants you to keep an eye on the machine for the administrative assistant to the CFO. Email logs show that there has been a spike in spam emails attempting to reach her email address. Even though she has completed the security awareness class, Tony doesn't want to take any chances. Tony hands you a Mandiant Analysis File to load into Redline and see if there is anything suspicious that is running, or was running, on her machine. After analysis, Tony, requires you to get a recent Mandiant Analysis File to analyze.

****Labs are only available in Full or Elite Editions of the course. To [ACCESS](#) your labs, go to the course in your members area and click the labs drop-down in the appropriate module line. To UPGRADE to gain access, click [LINK](#).***

Labs

Hunting Malware Part 2

Your manager, Tony, received 2 memory files from another facility within the ISAC. These 2 memory files were from actual incidents that took place within their facility a few years ago. Tony wants you to analyze them to see if you are able to analyze them for any signs of code injection and/or a rootkit to prepare you to detect APT attacks.

Hunting Empire

Your manager, Tony, wants to make sure that you can detect the widely used attacking tool, Empire. A hunting exercise has been scheduled, where you are tasked with detecting Empire's presence on an endpoint.

****Labs are only available in Full or Elite Editions of the course. To [ACCESS](#) your labs, go to the course in your members area and click the labs drop-down in the appropriate module line. To UPGRADE to gain access, click [LINK](#).***

